The generalized adjudication calculus

Version 1.38.

Scope. This document specifies **movement-phase adjudication** in full; retreats and builds I specify in outline (Sec. 9). It is the contract for both implementations: `adjudicator/csp.py` (the stable-labeling enumerator – the executable form of Sec. 5) and `adjudicator/resolver.py` (the staged fixpoint resolver – the algorithm of Sec. 6). The algorithm of Sec. 6 is my own five-phase design. The ruleset interface is frozen: two concrete rulesets back every axis.

Normative sources. The classic rulebook as tested by the DATC (live copy: webdiplomacy.net/datc.php) for the standard instantiation – classic Diplomacy on its classic map: *standard* is the engine's name for that ruleset and that map, and Sec. 8.1 defines the instantiation exactly – and `BeleriandInstruction.docx` (repo root) for the Beleriand instantiation. Kruijswijk's *Math of Adjudication* and logic-programming fixpoint semantics are related work, not blueprints. The DATC case ids referenced here as "(DATC 6.X: behavior)" are those of DATC v3.3; the cases are pinned in the case files under `tests/datc/`, and the choices among the DATC's alternative outcomes are listed in `docs/datc-choices.md`. A Beleriand case id BATC.X.n is case n of section X of the Beleriand adjudicator test cases, this project's own cases derived from the Beleriand rulebook (`tests/batc/cases/`, Sec. 10).

1 State, orders, order sheets

Let # be the counting measure.
Every set that is defined in the document is finite.

Let Powers be the set of powers.

Map. Let a map be a tuple $(P, \mathit{Loc}, \mathrm{prov}, \mathrm{terrain}, \mathrm{cap}, \mathrm{adj}_\mathrm{A}, \mathrm{adj}_\mathrm{F})$: a set of provinces P; a set of *locations* Loc with a function $\mathrm{prov}\colon \mathit{Loc} \to P$ – a location is a province together with an optional named coast, and prov forgets the coast (`src/entente/engine/map/types.py`); a *terrain* function $\mathrm{terrain}\colon P \to \{\mathrm{inland}, \mathrm{coastal}, \mathrm{sea}\}$; a *capacity* function $\mathrm{cap}\colon P \to \mathbb{N}^+ \cup \{\infty\}$; the *army adjacency* relation $\mathrm{adj}_\mathrm{A} \subseteq P \times P$; and the *fleet adjacency* relation $\mathrm{adj}_\mathrm{F} \subseteq \mathit{Loc} \times \mathit{Loc}$.
I write a province $p \in P$ also for its location that names no coast, so that $P \subseteq \mathit{Loc}$ and $\mathrm{prov}(p) = p$; a province with named coasts has one more location for each of them.
Both adjacencies are symmetric and neither joins a province to itself – $(p, p) \notin \mathrm{adj}_\mathrm{A}$ for every $p \in P$, and $\mathrm{prov}(l) \neq \mathrm{prov}(l')$ whenever $(l, l') \in \mathrm{adj}_\mathrm{F}$, so no two coasts of one province are joined either – and adj_A relates no province to a sea province. Say that two provinces $p, q \in P$ are *adjacent for fleets* iff $(l, l') \in \mathrm{adj}_\mathrm{F}$ for some $l, l' \in \mathit{Loc}$ with $\mathrm{prov}(l) = p$ and $\mathrm{prov}(l') = q$.
Let an *Area* be a province p with $\mathrm{cap}(p) > 1$: several units may stand in it at once.

Let an *unbounded Area* be an Area p with $\mathrm{cap}(p) = \infty$, and a *bounded Area* is any other. The name keeps apart what the rulebook files under one heading, "Provinces off the map": its two special Provinces off the map, Angband and Eriador, are the unbounded Areas of the Beleriand map, while its three off-map boxes have capacity 1 and are no Areas at all. The rulebook's own thirteen Areas are the bounded ones, of capacity 6. An unbounded Area is an Area in every rule of the calculus: entry into it displaces nobody, as entry into any Area displaces nobody; a Dislodge against a unit standing in it and a Bar given from inside it pass their capacity tests (Sec. 4.2), since each of the two orders acts on an Area alone – a Dislodge names a victim standing in one, and a Bar is given by a unit standing in one; and a unit standing in it may support every order whose target is that Area (the support rule, below). It differs from a bounded Area in two things: it never runs out of room, so the admission rule of Sec. 4.3 turns nobody away, and who may enter it at all is decided by the ruleset's *entry-permission* predicate (Sec. 4.3, Sec. 7), which reads the Area's entry restriction, below.

A map may carry data that only a ruleset reads. The Beleriand map carries a set $\mathit{Fort} \subseteq P$ of *fortresses*; a relation $\mathit{Pen} \subseteq P \times P$ of *penalized crossings*, where $(p, q) \in \mathit{Pen}$ says that the crossing from p into q climbs a mountain barrier in its marked direction, the reverse crossing (q, p) being penalized only where it is marked as well; a function $\mathrm{supply}\colon P \to \mathbb{N}$ counting the supply centres of a province (Angband holds seven); and, for each unbounded Area, its *entry restriction* $\in$ Powers, the one power whose units may stand in it, together with the fact that it is *one-way*: a unit may leave it, and no unit enters it from the board, the restricted power's units included, but for the exceptions its ruleset makes (Sec. 7: MORGOTH's units of base strength 3 may enter Angband, and nothing enters Eriador). Classic maps have $\mathrm{cap} \equiv 1$. Beleriand has eighty-four provinces: ten inland Areas and three sea Areas, all of capacity 6; the two unbounded Areas; three off-map boxes, inland, of capacity 1; and sixty-six provinces of capacity 1 that are no Areas – fifty inland and sixteen coastal, so no coastal province is an Area.

Game state. Let the game state be an immutable tuple $(\mathcal{U}, \mathrm{pow}, \mathrm{kind}, \mathrm{loc}, \mathrm{base}, \mathit{phase})$ on a map – immutable in that adjudication never changes a state: it reads one and yields the next.

Let $\mathcal{U}$ be the set of *units* on the board.

Let $\mathrm{pow}\colon \mathcal{U} \to$ Powers give the power a unit belongs to.

Let $\mathrm{kind}\colon \mathcal{U} \to \{\mathrm{army}, \mathrm{fleet}\}$ be the kind of a unit.

Let $\mathrm{loc}\colon \mathcal{U} \to \mathit{Loc}$ be a location where the unit is.

Let $\mathrm{base}\colon \mathcal{U} \to \mathbb{N}^+$ be a unit's *base strength*, its strength before any modifier and any support: 1 for every unit on the standard map, and 1, 2 or 3 in Beleriand (Sec. 7).

In Beleriand a *Double Unit* – a Double Army or a Double Fleet, in the rulebook's words – is a unit u with $\mathrm{base}(u) = 2$; the game layer binds it to a supply centre (Sec. 9).

Let $\mathit{phase} \in \mathbb{N} \times \{(\mathrm{S}, \mathrm{M}), (\mathrm{S}, \mathrm{R}), (\mathrm{F}, \mathrm{M}), (\mathrm{F}, \mathrm{R}), (\mathrm{W}, \mathrm{A})\}$ be the phase the state belongs to: a year, and which of the year's five phases it is – spring movement, spring retreat, fall movement, fall retreat and winter adjustment, in this order – written with the letters of the engine's phase tag, as in `S1901M`; the years count up from 1901 on the standard map and down from 25 to 0 in Beleriand.

None of the four functions is required to be injective: two different units may agree in power, kind, base strength and location, and they are still two elements of $\mathcal{U}$. The calculus needs values that differ between two such units – the order each received, whether each remains, whether each is dislodged – and a multiset could not carry them, since copies of one element of a multiset cannot be told apart. (The engine stores a unit as a value,

so two units that agree in everything are two equal values, which it tells apart by their position.)
Where units that cannot be told apart stand together in an Area, the rules bend for them; Sec. 4.4 says how, and until then the text speaks of units that can be told apart.
For every unit $u \in \mathcal{U}$ and $p \coloneqq \text{prov}(\text{loc}(u))$: $\text{terrain}(p) \neq \text{sea}$ and $\text{loc}(u) = p$ if $\text{kind}(u) = \text{army}$, and $\text{terrain}(p) \neq \text{inland}$ and $\Big(\text{loc}(u) = p \iff \#(\text{prov}^{-1}(p)) = 1\Big)$ if $\text{kind}(u) = \text{fleet}$, where $\#(\text{prov}^{-1}(p)) = 1$ iff p has no named coast.
For a province $p \in P$, let $\text{occupants}(p) \coloneqq \{u \in \mathcal{U} \mid \text{prov}(\text{loc}(u)) = p\}$; every state satisfies $\#(\text{occupants}(p)) \leq \text{cap}(p)$ for every $p \in P$.
The state holds no ownership: the game keeps the ownership of the supply centres beside it, movement adjudication never reads it, and the adjustment of Sec. 9 does.

Let each power $\pi \in \text{Powers}$ submit an ordered list of orders – a *sheet* σ_π. Order sheets are ordered. Sequence is semantically load-bearing in Beleriand (Sec. 4.2: which of a power's Bars against one entrant stands; Sec. 4.4: which unit an order or a reference binds when several fit it); the standard ruleset reads it only to take the first legal one of several orders written for one unit (DATC 4.D.3; Phase 0, Sec. 6) and, in the adjustment, to let the first legal orders of a sheet use up the power's quota (DATC 4.D.4, 4.D.5 and 4.D.6, choice B; Sec. 9), and for nothing else.
Units without orders hold.
Let the *canonical order* be the order of provinces by their keys, of powers by their names, and of units by province key, then coast, power, kind and base strength; a unit's *canonical index* is its place in the last, and so is that of the order it received.

Written forms. In a written form, `<·>` is a placeholder for the part it names, and `[·]` is an optional part; everything else is typed as it stands.
Let `p` be any location.
Let a *unit spec* `u` be the text

```
[<strength>]<kind>[(<designator>)] <location>
```

where `<strength>` is a digit from 2 to 9, written exactly when the unit's base strength is not 1; `<kind>` is `A` (army) or `F` (fleet); `<designator>` is `M` or `G`, written exactly when the unit is one of Beleriand's two special units ("Special-unit designators" below); and `<location>` is a province key with an optional named coast. For example: `A PAR`, `F STP/SC`, `2A GEL`, `3A(M) ANF`.
Let a *unit reference* be the text `<power> u`: a unit named together with its owner, the way a Dislodge or a Bar (the order registry, below) names its victim – for example `MORGOTH A NEB`.
An unwritten strength prefix means base strength 1 in every unit spec and reference: `A` is the own name of an army of base strength 1 and never a wildcard (the rulebook: "A Unit type written with no Strength value before it always means Strength 1: "A" is the same as "1A", and never stands for an Army of any other Strength"; on the standard map every unit has base strength 1, so the reading changes nothing there). It is also that army's only name: the parsers reject `1A`.
A wrongly written *kind* or base strength in an order's own unit spec, or in the spec of a support's subject or a convoy's cargo, is advisory on the standard map – the order still binds the unit standing in the province, and the support or the convoy its subject (DATC 4.C.2 choice b, case 6.D.35) – and makes the order illegal in Beleriand (Sec. 7, `strict_designation`). A Disband of the adjustment phase follows the same rule for

its written base strength, its written kind being read on neither map: `Disband 2A PAR` written for `A PAR` removes that army on the standard map and binds no unit in Beleriand (Sec. 9). A Dislodge's or a Bar's victim reference is read exactly: written with a wrong kind or base strength it describes no unit, and the order stays valid and fails (Sec. 3.4). But a Bar whose reference writes a kind that could not reach the barring unit's Area from the province that the reference writes fails its own static check, and it is illegal (below). Neither order is in the standard registry (the order registry, below), so under the standard ruleset a Dislodge or a Bar is illegal and ignored, whatever its reference says.

Order registry (per ruleset). There are six order types. A *Hold* keeps its unit where it stands; a *Move* has a *destination*, a location its unit tries to enter; a *Support* has a *subject*, an order of another unit, to which it adds its strength; a *Convoy* carries a Move of an army across the sea province its fleet stands in; a *Dislodge* names a *victim* by a unit reference – a unit standing in an Area – tries to displace it, and enters that Area if it succeeds; a *Bar* names a victim by a unit reference too, and denies it entry into the Area the barring unit stands in. Hold, Move, Support and Convoy are the core of both rulesets; Dislodge and Bar are Beleriand's.
An order is *relocating* if it is a Move or a Dislodge, and *stationary* otherwise. A relocating order makes its unit an *attacker* in the sense of support cutting (Sec. 3) and of the fortress rule (Sec. 2); a Dislodge relocates like a Move, with a named victim – it enters the victim's Area if it succeeds.
A support's subject is a whole order (Beleriand writes it out; standard support-move/support-hold are the two core cases). A support's *target* is the province its subject acts on: the supported Move's destination, the supported Dislodge's victim Area, the supported Bar's barred Area, and for a support-hold the province of the supported unit.
Movement-phase order types, in the engine's text notation:

Type	Core/variant	Written form	Effect class
Hold	core	`u H`	stationary
Move	core	`u - p[ via convoy]`	relocating; contests entry at `p`
Support	core	`u S [<power> ]<subject order>`	stationary; adds strength to its subject
Convoy	core	`u C <move>`	stationary; carries a move's path
Dislodge	Beleriand	`u DIS <power> <victim>[ <victim's action>]`	relocating; targeted displacement and entry
Bar	Beleriand	`u BAR <power> <victim> - p`	stationary; targeted entry denial

`format_order` writes every order in this notation and the parsers invert it exactly (`src/entente/engine/orders.py`), and this document writes its orders in it. The parsers ignore case and accept a Hold with its `H` left off. A support's subject order and a convoy's move are written as their own units would write them, less two parts – a supported Hold drops its `H`, and neither carries `via convoy`: `A SMY S F ANK`, `F CON S F BLA - ANK`, `F NTH C A LON - HOL`. A subject order is a Hold, a Move, a Dislodge or a Bar: the parsers reject a subject written as a Support or a Convoy, such as `A SMY S F ANK S A CON`, and a unit ordered to support or to convoy is backed by a support-hold, `A SMY S F ANK`, which matches every stationary order (below). A support may write its subject's owner `<power>`, which an Area can need to tell units apart: `A EST S MORGOTH A NEB`. A Dislodge's `<victim's action>` is the victim's order without its unit spec, which the reference has already named: `A EST DIS MORGOTH A NEB S A GEL - THA` names MORGOTH's `A`

NEB as its victim and A NEB S A GEL - THA as the victim's order (Sec. 3.4). A Bar's - p is the victim's move into the Area in which the barring unit stands: A NEB BAR DORIATH A FOR - NEB.
The rulebook writes the same orders in its own notation: a unit spec as XA(Province) – 2A(GEL), 3A(M)(ANF) – the order words in full or shortened to sup, cnv, dis, bar (a Hold is holds, a support-hold sup hold), a move's dash with no spaces around it, and an owner in the possessive. Its A(Est) dis Morgoth's A(NEB) sup A(Gel)-Tha is A EST DIS MORGOTH A NEB S A GEL - THA in the engine's notation. The other two phases write orders of their own (Sec. 9): a retreat is a plain Move u - p or Disband u; an adjustment is Build u, Disband u or Waive, and in Beleriand the build of a Double Unit may name the supply centre it binds, Build u BOUND c with c a province key – Build 2A NAR BOUND RON.

A support s *matches* an order o that a unit u received iff u is the unit that the subject reference of s describes – by province, by owner where one is written, and in Beleriand by designator ("Special-unit designators", below), kind and base strength as well, the kind and the base strength being advisory on the standard map ("Written forms", above) and the designator meaningless there – and never the supporting unit itself, and o is the subject order of s: a Move to the written destination province, whatever its route, and whatever its coast where the support writes none, the same coast where the support writes one; a Dislodge, or a Bar, naming the same victim – owner, kind, base strength and, for a special, its designator – and, for a Bar, the same Area; a support-hold matches every stationary order of u. So a support-hold whose subject was ordered to move matches nothing, and neither does a support written for a Dislodge or a Bar against 3A(G) when the subject's order names 3A(M): the two name different units, and the support names an order nobody gave. A legal support that matches no order is invalid and becomes a Hold (below), so it never succeeds. A support whose subject is not there, or whose subject reference is ambiguous, matches no order either, but it is illegal (below): it is ignored, and its unit takes its next legal order, or holds. The order a unit received is the one Phase 0 leaves it (Sec. 6), a Hold where the order written for it is invalid or where every order written for it is illegal: a support of a Move, a Dislodge or a Bar that its unit never received matches nothing, and a support-hold matches the Hold the unit keeps. A legal Move written via convoy that no fleet is ordered to convoy is no Hold: it stays a relocating order, and it fails (Sec. 3.5). A subject reference never describes the supporting unit itself – a unit cannot support its own order – so the supporting unit is excluded before the ambiguity rules of Phase 0 (Sec. 6) apply, and in an Area a support may name the order of a unit standing beside it (an Area-mate's move out; rulebook ruling, BATC.B.13/B.14).
Say that a unit u *reaches* a province $q \in P$ iff it could relocate there by its own adjacency: $(\mathrm{prov}(\mathrm{loc}(u)), q) \in \mathrm{adj}_{\mathrm{A}}$ if u is an army, and $(\mathrm{loc}(u), l) \in \mathrm{adj}_{\mathrm{F}}$ for some $l \in \mathit{Loc}$ with $\mathrm{prov}(l) = q$ if u is a fleet. A support is legal when its supporter reaches the target (the classic rule: a unit supports only where it could have gone) **or** when it stands in the target and the target is an Area. A unit never reaches the province it stands in, so the second clause is the whole of what a unit may back inside its own Area, and the rulebook grants all of it: "a Unit can Support any Unit of its own Area, and can Support any Order which targets the Area in which it itself stands". What the unit may back elsewhere the reach rule decides, as for every unit. Four subject shapes target an Area: an Area-mate's Bar and an Area-mate's stationary order backed by a support-hold, both ordered from inside, and a Move *entering* the Area and a Dislodge whose victim stands in it, both ordered from outside – a Move into a province is written by a unit standing elsewhere, and a

Dislodge is never written from inside its victim's Area, since its unit has to reach and enter that Area. So the second clause, not co-location, is what admits a support from inside an Area; whether the subject is an Area-mate decides nothing. An Area-mate's Move *out* needs the first clause alone: its target lies elsewhere, and the supporter has to reach it, as every supporter has. Where the target is no Area – capacity 1, which is every province of the standard map – only the reach rule applies, so a unit cannot back an attack on the province it occupies alone (DATC 6.D.34). A unit is never its own subject, in an Area or out of one. So a unit standing in an Area may back a Dislodge whose victim it is itself: the subject is the dislodger's order and the target is the supporter's own Area, and the second clause admits here what the reach rule refuses at capacity 1 (DATC 6.D.34). Such a support counts toward the Dislodge's prevent and never toward its attack (the role filter, Sec. 3.3).

Illegal orders are ignored, invalid orders become Hold. The first is both rulebooks' rule (Standard – DATC 4.E.1 choice; Beleriand – "An illegal Order is ignored: if the same Unit is given another, legal Order, that Order is executed, and otherwise the Unit holds", whose "Invalid Orders" are the orders this document calls illegal: "Support for a Move to a Province that is not adjacent"). The second is this document's: an invalid order keeps its unit, which holds, and no later order on the sheet is consulted for that unit. An order is *illegal* where the board as it stands forbids it, read without any other order – a destination that its unit does not reach, unless the order is an army's Move that is a convoy attempt ("convoyed", the Notation paragraph below), terrain-illegal moves, an entry the ruleset's entry permission refuses, a supporter that neither reaches its target nor stands in it where the target is an Area, a subject or a cargo that is not there, a Convoy given by a unit that is not a fleet standing in a sea province, or whose cargo is no army, or whose cargo its own fleet may not carry (the capability gate of Sec. 3.5), an ambiguous unit reference (the rulebook's choice for DATC 4.C.5 and 4.C.6) – and an illegal order is ignored, so that the unit takes its next legal order, or holds (Phase 0, Sec. 6). The rulebook's choice for those two cases reads "If a specification of Unit is ambiguous, then the Order is changed into Hold Order (like any other invalid Order)", and its invalid Orders are the orders this document calls illegal (above), which give way to the unit's next legal order. In a Beleriand game the entry permission into `GON` reads an order: the shield that closes `GON` to every power but GONDOLIN falls in the phase in which GONDOLIN writes a Move or a Dislodge for the Double Army standing there, whether Phase 0 finds that order illegal, passes it over or keeps it, and whether or not it succeeds (the rulebook's "Gondolin"). The shield falls as well once a unit of MEN stands in a province adjacent to `GON`, or once the Double Army has left `GON` by any other way, and it never returns (the rulebook: "Once lost, this protection never returns"). So there the legality of another power's entry into `GON`, which the entry-permission predicate decides (Sec. 7), depends on what GONDOLIN writes. Among the static checks, a Dislodge passes its own iff its victim reference writes a power other than its unit's own – no power dislodges its own unit (the rulebook: "no Power may Dislodge its own Unit") – and a province that is an Area other than its unit's own, which its unit reaches and may enter; and a Bar passes its own iff its unit stands in an Area, its `- p` names that Area, and a unit of the kind that its victim reference writes would reach that Area from the province that the reference writes. Whether a unit standing there answers to the reference is no static matter: a reference that describes no unit leaves the order valid, and the order fails (Sec. 3.4, Sec. 4.2). A Move written `via convoy` passes its own iff its unit is an army and both the province it stands in and its destination are coastal (the rulebook: "If moving between two Coastal Provinces, a Player must specify

if the Unit Moves via Convoy"). So a fleet's Move written so, or an army's from or into a province that is not coastal, is illegal, although the same Move written without `via convoy` may be legal; an army's between two coastal provinces is legal whatever fleets stand on the board, and it fails where no chain was ordered (Sec. 3.5). Coasts, which only the standard map has, are read by the DATC choices 4.B.1, 4.B.2, 4.B.3, 4.B.5 and 4.B.6 listed in `docs/datc-choices.md`. An order binds its unit by province and relocates it from the location where the unit stands, whatever coast the order's own spec writes; an army's Move enters the province it names, any written coast dropped; a fleet's Move that writes no coast into a province with named coasts goes to the coast its fleet reaches where it reaches exactly one, and is illegal otherwise; and a fleet's Move to a coast it does not reach is illegal. A legal order is *invalid* where it fails what it reads from the other orders – support for an order the subject did not receive, a convoy for a Move the cargo was not ordered – and an invalid order keeps its unit and becomes a Hold. Before the adjudicator runs, the Beleriand game layer also voids every Support that one of its four support bans forbids (`src/entente/variants/beleriand/layer2.py`). The bans are the rulebook's "None of the elven power can support or be supported by Morgoth.", read in both directions as two bans, one on an elven power supporting a MORGOTH unit and one on MORGOTH supporting an elven unit; its "No other elven Powers may Support or be Supported by Feanorian Powers."; and its "Any Power with Unit in Anfauglith may not be Supported by Feanorian Powers." A Support whose subject is the order of a unit of the supporter's own power is never banned, the Supports that MEN and NAUGRIM give are bound by none of the bans, and no Convoy is. Like an illegal order, a banned Support is ignored, and the unit takes its next order or holds. The game layer also discards every order of a defeated MORGOTH, whose units hold; voids every order written for the TELERI army that stands in disorder (the rulebook's "Teleri"), which holds – where that army has identical twins, the orders Phase 0 binds to it, so that each other twin keeps the order Phase 0 gives it; and withholds its own orders – Transform, Devastate, Make-Masks, the mask transfers and Sail-West – which it resolves after the adjudicator, so that a unit whose only orders are these is unordered to the adjudicator and holds. On the standard map such an order fails, and the unit takes its next order or holds. An `OrderResolution` may carry an optional `reason`: the sentence of the rule that voided the order, for a failure the board cannot show. Beleriand's four support bans populate it; everything else leaves it absent. It is presentation only: the calculus never reads it, and a row's `Resolution` is what it would be with the field gone, so it changes no outcome.

Special-unit designators (Beleriand). A unit spec in order text names one of MORGOTH's two special units – the Morgoth unit or Glaurung, the only units of base strength 3 the game has – with its *designator*, `(M)` or `(G)` – the rulebook's own notation – written immediately after the strength and kind token: `3A(M) ANF - LOT`, `A FOR DIS MORGOTH 3A(G) ANF`. The designator is required: a special is named only with it, in its own orders and wherever another order names it – a Dislodge's or a Bar's victim, a support's subject, a convoy's cargo – and a `3A` written without one fits no unit (the rulebook's Signature rule). On the board the two agree in power, kind and base strength – both are `3A` – so the designator is the only thing that tells them apart for the adjudicator; which of them is which is recorded in the game layer's unit ledger (`src/entente/variants/beleriand/ledger.py`), and the game layer writes it into every order it lists. A reference has to match a designator exactly, both ways: `A FOR DIS MORGOTH 3A(G) ANF` names Glaurung and nobody else, and a reference written without a designator fits only a unit whose order writes none. A special keeps its designator whatever its order says: an invalid order of a

special degrades to a Hold that carries the designator, and a special for which no valid order was written holds under its designator all the same, the game layer supplying that Hold to the adjudicator. The Beleriand game layer enforces the rule at every position named above: an order whose own spec or whose reference describes a special without its designator, or writes a designator naming no special standing at the written spec, is ignored, like one with a wrong unit type, and the unit takes its next order or holds; and the designator narrows which *identity* the order binds – which special moves, retreats, or goes to limbo. The adjudicator knows a special only by the designator its own order writes, so a `3A` whose order writes none is, to it, an unnamed `3A` – no special – which only a bare reference fits; in a game every special's order carries its designator. On the standard map a designator means nothing: nothing refuses it, and wherever an order carries one, matching ignores it, so `A(M) PAR - BUR` is adjudicated as `A PAR - BUR` is, and both `A MAR S A PAR - BUR` and `A MAR S A(M) PAR - BUR` match it (Sec. 7, `strict_designation`). The adjudicator reads the designator wherever a reference is bound to a unit or to an order – a Dislodge's victim (Sec. 3.4), a Bar's victim (Sec. 4.2), a support's subject ("matches", above) – and Phase 0 reads it when it decides whether a support matches an order at all: the subject's designator, and for a support-dislodge or a support-bar the victim's too; the strength arithmetic never reads it: a designator decides which unit or order a reference binds, never a number. A special's own spec it reads like any other unit's: the order claims a unit of its power, kind and base strength in its province, and which of the two specials that unit is stays the ledger's business. Where both specials stand in one Area, an order whose own spec writes the designator that an earlier legal order of its sheet has already written there is passed over, like any second order written for one unit (Phase 0, Sec. 6), so that the other special keeps its own order or the Hold that the game layer supplies. `legal_orders` writes every special with its designator, in its own orders and in every order that names it; where both specials stand in one Area they are two units with two sets of orders.

Notation. Let $\mathcal{O}$ be the set of orders of the phase as Phase 0 leaves them (Sec. 6): every unit has exactly one, a Hold where no order was written for it or the one written was invalid.
Let $\mathrm{unit}\colon \mathcal{O} \to \mathcal{U}$ give the unit that received an order; it is a bijection, and for a unit $u \in \mathcal{U}$ let $o_u \coloneqq \mathrm{unit}^{-1}(u)$ be the order it received.
For an order $o \in \mathcal{O}$, I write $\mathrm{prov}(o)$ for $\mathrm{prov}(\mathrm{loc}(\mathrm{unit}(o)))$, the province in which its unit stands.
Let $\mathcal{M} \subseteq \mathcal{O}$ be the set of relocating orders and $\mathcal{S} \subseteq \mathcal{O}$ the set of support orders.
Say that the fleets standing on the board *could carry* an army from a province p to a province p' iff there is a sequence $(q_1, \ldots, q_k)$ of sea provinces, $k \geq 1$, with q_1 adjacent for fleets to p, q_i to q_{i+1} for every $i < k$ and q_k to p', every q_i holding a fleet, of any power and whatever its order, that may carry that army (the capability gate of Sec. 3.5).
A Move $m \in \mathcal{M}$ is *convoyed* iff its army is to travel by sea – the Move is written `via convoy`, or it names a coastal province the army does not reach and the fleets standing on the board could carry it there, so that Phase 0 makes it a convoy attempt (DATC 4.E.1) – and *overland* otherwise; a Dislodge is never convoyed, so it is overland: it has no `via convoy` form, and its unit has to reach the victim's Area.
Let $\mathrm{dest}\colon \mathcal{M} \to P$ give the province a relocating order enters if it succeeds: a Move's destination province, a Dislodge's victim's Area.
Let $\mathrm{target}\colon \mathcal{S} \to P$ give a support's target (above).
Say that a Convoy c is a *Convoy of* a convoyed Move $m \in \mathcal{M}$ iff $\mathrm{unit}(m)$ is the unit that

the cargo spec of c describes – by province, and in Beleriand by kind and base strength as well, which the standard map reads as advisory ("Written forms", above) – and the destination province written in c is $\mathrm{dest}(m)$, whatever coast either writes. A legal Convoy that is a Convoy of no convoyed Move is invalid and becomes a Hold (above).
Say that an order $o \in \mathcal{O}$ is *aimed at* a province $p \in P$ iff o is a relocating order with $\mathrm{dest}(o) = p$, a support with $\mathrm{target}(o) = p$, a Convoy of a Move $m \in \mathcal{M}$ with $\mathrm{dest}(m) = p$, or a Bar with $\mathrm{prov}(o) = p$ – a Bar guards an entry into its own Area.
Say that a support $s \in \mathcal{S}$ is *directed against* the unit its subject attacks or bars: for a supported Move whose destination has capacity 1, the unit standing in that destination, if any (a Move into an Area attacks nobody); for a supported Dislodge, the victim it binds (Sec. 3.4), nobody where it binds none; for a supported Bar, the entrant the Bar binds (Sec. 4.2), nobody where it binds none; a support-hold is directed against nobody. A Bar is directed against the entrant it binds.
For a relocating or Bar order $o \in \mathcal{O}$, let $\mathrm{supp}(o) \subseteq \mathcal{S}$ be the set of supports that match o and are no support-holds, and for a unit $u \in \mathcal{U}$ whose order o_u is stationary let $\mathrm{hsupp}(u) \subseteq \mathcal{S}$ be the set of support-holds that match o_u:

$$\mathrm{supp}(o) \coloneqq \{\, s \in \mathcal{S} : s \text{ is no support-hold and } s \text{ matches } o \,\},$$
$$\mathrm{hsupp}(u) \coloneqq \{\, s \in \mathcal{S} : s \text{ is a support-hold and } s \text{ matches } o_u \,\}.$$

All strength arithmetic is exact rational arithmetic (`fractions.Fraction`); no floats, anywhere.

2 The strength pipeline

Every contest of Sec. 4 consumes strengths produced by one pipeline. I fix the types first and define each function before its first use; the two inputs that can only be defined later are named as such at the place where they enter.

Strength roles. Let $\mathrm{Roles} \coloneqq \{\mathsf{move}, \mathsf{support}, \mathsf{hold}\}$ be the set of *strength roles* (`StrengthRole` in the code): move is a relocating order acting on the province it is aimed at, support is a support order contributing to its subject, and hold is a unit with a stationary order resisting in its own province.
A strength role is a label and is written in sans serif. It is never one of the contest functions of Sec. 4, which are set upright: hold is an argument of mods, while hold is a function of a unit.

Modifiers (mods**).** Let $\mathrm{mods} \colon \mathrm{Roles} \times \mathcal{O} \to \mathbb{Q}^+$ be the *modified base*: $\mathrm{mods}(\rho, o)$ is the base strength $\mathrm{base}(\mathrm{unit}(o))$ of the order's unit after the ruleset's modifiers for the order o in the role ρ have been applied.
Modifiers are composable ruleset data and they are *static*: mods reads the map and the order sheets, and never a resolution (`Ruleset.modified_base`). The two rulesets define it as follows.

- Standard – there are no modifiers: $\mathrm{mods}(\rho, o) \coloneqq \mathrm{base}(\mathrm{unit}(o))$ for every role ρ and every order o.
- Beleriand, *mountain barriers* – an order whose crossing is penalized has its strength halved. For a relocating order o that is not convoyed, $\mathrm{mods}(\mathsf{move}, o) \coloneqq$

$\frac{1}{2}\,\mathrm{base}(\mathrm{unit}(o))$ if $(\mathrm{prov}(o), \mathrm{dest}(o)) \in \mathit{Pen}$: its unit *crosses* a barrier. For a support s, $\mathrm{mods}(\mathsf{support}, s) \coloneqq \frac{1}{2}\,\mathrm{base}(\mathrm{unit}(s))$ if $(\mathrm{prov}(s), \mathrm{target}(s)) \in \mathit{Pen}$: its unit *supports through* a barrier, whatever the shape of its subject. The crossing in the other direction is penalized only where *Pen* says so, and $\mathrm{mods}(\mathsf{hold}, o)$ is never halved, so a unit's strength for retaining its own province is unaffected (directional).

- Beleriand, *the convoy toll* – a convoyed Move never walks its origin edge; its toll is the crossing at which it disembarks (the rulebook: "A Convoyed Army does not pay for the Province it left - it pays for its disembarking: if the crossing from the last Sea of the Convoy into the landing Province is marked, the Move is halved"). The last sea belongs to the convoy and not to the coast, so the toll reads the convoy as the sheets order it.
  Let $o \in \mathcal{M}$ be a convoyed Move, $p \coloneqq \mathrm{prov}(o)$ and $p' \coloneqq \mathrm{dest}(o)$. Call a sea province q adjacent for fleets to p' *penalized* iff $(q, p') \in \mathit{Pen}$, and *free* otherwise. Let an *ordered chain* of o be a sequence $(q_1, \ldots, q_k)$ of sea provinces, $k \geq 1$, such that q_1 is adjacent for fleets to p, q_i to q_{i+1} for every $i < k$, and q_k to p', and such that every q_i holds a fleet whose order in $\mathcal{O}$ is a Convoy of o. These are the chains of Sec. 3.5 with one condition left out: no fleet's dislodgement is read.
  Let the *ordered disembark seas* of o be the set of the last seas of its ordered chains,

$$\mathrm{disembark}(o) \coloneqq \{\, q_k : (q_1, \ldots, q_k) \text{ is an ordered chain of } o \,\}.$$

  Call o *tolled* iff $\mathrm{disembark}(o) \neq \emptyset$ and every sea in $\mathrm{disembark}(o)$ is penalized. Then

$$\mathrm{mods}(\mathsf{move}, o) \coloneqq \begin{cases} \frac{1}{2}\,\mathrm{base}(\mathrm{unit}(o)) & \text{if } o \text{ is tolled,} \\ \mathrm{base}(\mathrm{unit}(o)) & \text{otherwise.} \end{cases}$$

  A sea counts only through a chain: a fleet ordered to convoy o from a sea that no ordered chain runs through opens no beach, and a sea adjacent for fleets to p' that holds no such fleet is not read at all. If $\mathrm{disembark}(o) = \emptyset$, no chain was ordered; the convoy then carries nothing whatever becomes of its fleets (Sec. 3.5), and no contest reads the strength of o.
  $\mathrm{disembark}(o)$ is a function of the order sheets alone, so mods stays static. The definition has one consequence that the rulebook's sentence does not have: an ordered chain counts whether or not it survives the turn. If ordered chains of o end in a penalized sea and in a free one, every chain that ends in a free sea loses a fleet to a dislodgement, and some chain that ends in a penalized sea survives, then the army goes ashore across a penalized crossing at its full strength, where the rulebook's letter – the last sea of the convoy that sailed – halves it. The two can part only at a *mixed coast*, a province with both a penalized and a free sea adjacent for fleets to it: everywhere else all the chains of o, ordered or surviving, end in seas of one kind. No shipped map has a mixed coast – the standard map has no barriers, and on the Beleriand map the tolled landing coasts ELO, NEV and VIN each touch exactly one sea (NBE) – so on the shipped maps disembark reads the rulebook's letter exactly, and a test holds every shipped map to that condition.
  Left out on purpose. A toll that reads the *surviving* chains of o – the rulebook's letter in full – is possible, and I leave it out on purpose. It would make mods read a resolution: the strength of o would depend on which convoying fleets are dislodged, their dislodgement on the supports behind the attacks on them, and those supports

– through the numeric cut of Sec. 3.2, in which $\mathrm{mods}(\mathsf{move}, o)$ is a summand – on the strength of o again. That is one more circularity source beside those of Sec. 5.2, with a backup rule of its own to settle; the cut of Sec. 3.2 would stop being a closed-form function of the order sheets and the convoy viabilities; and the resolver of Sec. 6 would have to carry the base of such a Move as an interval until the fates of its fleets are decided. None of this would change an outcome on any map that exists, because none has a mixed coast: the price is a more complex algorithm, and the gain in every existing scenario is nothing. So the calculus decides statically what the order sheets can decide and leaves the rest, and a map with a mixed coast would reopen that choice.

- Beleriand, *fortresses* – $\mathrm{mods}(\mathsf{hold}, o) \coloneqq \mathrm{base}(\mathrm{unit}(o)) + 1$ for an order o that is not relocating whose unit stands in a fortress, $\mathrm{prov}(o) \in \mathit{Fort}$ (a failed Move or Dislodge forfeits the bonus); the bonus enters $\mathrm{mods}(\mathsf{hold}, \cdot)$ alone, never $\mathrm{mods}(\mathsf{move}, \cdot)$ or $\mathrm{mods}(\mathsf{support}, \cdot)$. No Bar is ever in a fortress: a Bar is given by a unit standing in an Area (Sec. 1), and no fortress of the Beleriand map is an Area. Were one there, its strength would still take no +1, being its unit's bare base plus its supports (Sec. 4.2); the +1 would go only to its unit's defence, through $\mathrm{mods}(\mathsf{hold}, \cdot)$.

Where no clause above applies, $\mathrm{mods}(\rho, o) \coloneqq \mathrm{base}(\mathrm{unit}(o))$.

Effectiveness (eff**).** Let $\mathrm{cut}\colon \mathcal{S} \to \mathbb{Q}_{\geq 0}$ be the *cut* of a support: the combined strength of the attacks that cut it. This is the first of the two later inputs: its definition needs the qualifying cutters and is the whole content of Sec. 3.2. To read the present section it is enough to know that $\mathrm{cut}(s)$ is a nonnegative rational fixed by the order sheets and the convoy viabilities.
For every support $s \in \mathcal{S}$, define its *effectiveness*

$$\mathrm{eff}(s) \coloneqq \max\Big(0,\ \mathrm{mods}(\mathsf{support}, s) - \mathrm{cut}(s)\Big),$$

the part of the support's modified base that survives its cut, so that $\mathrm{eff}\colon \mathcal{S} \to \mathbb{Q}_{\geq 0}$.
A support gives its effectiveness to its subject only while its own unit stands, with one exception: a support of a Bar gives it whatever becomes of its unit (the rulebook: "A Support given to a Barring Order is cut as any other Support, but it is not lost when the Unit giving it is Dislodged in the same Phase"; Sec. 3.3).
Say that a support $s \in \mathcal{S}$ *backs a Bar* iff $s \in \mathrm{supp}(b)$ for some Bar order $b \in \mathcal{O}$. A support-hold whose subject reference describes the unit of a Bar b matches b (Sec. 1) and backs no Bar all the same: it lies in $\mathrm{hsupp}(\mathrm{unit}(b))$ – in a stack, in the walls of Sec. 4.4 – and not in $\mathrm{supp}(b)$, so it adds to the defence of $\mathrm{unit}(b)$ and nothing to the strength of b (Sec. 4.2), and it delivers nothing once its giver is dislodged. A support of a Bar has to say so (the rulebook: "In case of Supporting a Barring Order, the Player must specify that the Barring Order is Supported (see below: Other Support Orders) and cannot use Support Hold Order!").
For every support $s \in \mathcal{S}$, define its *delivered effectiveness*

$$\mathrm{delivered}(s) \coloneqq \begin{cases} \mathrm{eff}(s) & \text{if } s \text{ backs a Bar,} \\ 0 & \text{if } s \text{ backs no Bar and } \mathrm{unit}(s) \text{ is dislodged,} \\ \mathrm{eff}(s) & \text{otherwise,} \end{cases}$$

so that $\mathrm{delivered}\colon \mathcal{S} \to \mathbb{Q}_{\geq 0}$. "Is dislodged" is the second later input: a unit is dislodged by a displacement contest (Sec. 4.2), and those contests consume the very strengths defined

here. The circle is real, and Sec. 5 resolves it by defining adjudication as a fixpoint; until then read "unit(s) is dislodged" as a given fact about each unit.
For a support that backs a Bar that fact is not read at all: there delivered(s) = eff(s) is, like cut(s), fixed by the order sheets and the convoy viabilities. No standard order is a Bar, so in the standard instantiation the first case never applies.

Value (value**).** For a base $x \in \mathbb{Q}_{\geq 0}$ and a set of supports $F \subseteq \mathcal{S}$, define the *pipeline value*

$$\text{value}(x, F) := x + \sum_{s \in F} \text{delivered}(s),$$

a base plus what the supports in F deliver, so that value: $\mathbb{Q}_{\geq 0} \times 2^{\mathcal{S}} \to \mathbb{Q}_{\geq 0}$. Every strength a contest consumes is value(x, F) for a base x and a set F of matching supports (Sec. 1); x is a modified base except in the two places where Sec. 4 says the bare base applies (the hold of a unit whose relocating order failed, and the strength of a Bar). Which base and which supports is what tells the four contest roles hold, attack, defend and prevent apart, and Sec. 4 defines them one by one: the supports of a relocating order feed its attack, defend and prevent, and the support-holds of a unit feed its hold. The one role-dependent filter – supports that cannot help dislodge their own power's unit – is in Sec. 3.3.

3 Support cutting – the unified numeric one-shot model

Both instantiations run the **same cut arithmetic**; they differ only in the *qualifying-cutter* predicate and the *target-selection* rule. This unification is deliberate: Lemma L2 (Sec. 8.1) shows the classic binary rule is exactly this model at base $\equiv 1$.

3.1 Qualifying cutters

For a support $s \in \mathcal{S}$, let the *qualifying cutters* QC(s) $\subseteq \mathcal{M}$ be the set of relocating orders a such that:

1. a *targets* unit(s): at capacity 1, dest(a) = prov(s); in an Area, a is a Dislodge whose victim is unit(s) (Sec. 3.4);
2. pow(unit(a)) $\neq$ pow(unit(s)) – own-power attacks never cut;
3. **the exception**: a is not the *protected attacker* of s (both rulesets: Standard – every attack from the province of the unit s is directed against, the destination of the supported Move, is protected; Beleriand – per **unit**: the one unit s is directed against is protected, and a *different* unit attacking from the same Area does cut). In both rulesets the route does not matter: an attack from the protected province, or by the protected unit, is protected *even when it arrives by convoy* (DATC 4.A.4 choice A). A support-hold is directed against nobody and protects nothing;
4. a *exists*: an overland attacker exists unless it is a mis-specified Dislodge (Sec. 3.4); a convoyed attacker exists iff its convoy is viable (Sec. 3.5). This is the *only* resolution-dependent input to cutting, and the reason cutting cannot be a standalone stage

when convoys are in play (Theorem T4, Sec. 8.4). It is also the door through which the classic convoy paradox enters the calculus: the cut a convoyed attack delivers can decide the sea battle that decides that same convoy's viability. Where this loop leaves a viability undecided, the **Szykman rule** settles it – the convoyed move fails, does not exist as an attacker and delivers no cut (Sec. 5.2).

3.2 One-shot simultaneous arithmetic

I build the cut bottom-up, in three definitions; each holds for every order of the kind it names.
For every support $s' \in \mathcal{S}$, define its *pre-cut value*

$$\mathrm{eff}_0(s') \coloneqq \mathrm{mods}(\mathsf{support}, s'),$$

the support's modified base before any cut and before any void, so that $\mathrm{eff}_0 \colon \mathcal{S} \to \mathbb{Q}^+$.
For every relocating order $a \in \mathcal{M}$, define its *cut value*

$$\mathrm{cutval}(a) \coloneqq \mathrm{mods}(\mathsf{move}, a) + \sum_{s' \in \mathrm{supp}(a)} \mathrm{eff}_0(s'),$$

the attacker's modified base plus the pre-cut values of all the supports that match it, so that $\mathrm{cutval} \colon \mathcal{M} \to \mathbb{Q}^+$.
For every support $s \in \mathcal{S}$, define its *cut*

$$\mathrm{cut}(s) \coloneqq \sum_{a \in \mathrm{QC}(s)} \mathrm{cutval}(a),$$

the combined cut value of its qualifying cutters, so that $\mathrm{cut} \colon \mathcal{S} \to \mathbb{Q}_{\geq 0}$ and $\mathrm{cut}(s) = 0$ when $\mathrm{QC}(s) = \emptyset$.
This is the cut that Sec. 2 subtracts: $\mathrm{eff}(s) = \max\Big(0,\ \mathrm{eff}_0(s) - \mathrm{cut}(s)\Big)$.

Note that all cuts are computed **simultaneously at pre-cut strengths**: every support counts toward cutting at its full modified base value *even if it is itself cut, or its unit is later dislodged*. This is the Beleriand rulebook's rule ("Strength of EVERY Support counts to the Support cutting, even if the Support is to be cut") and it is what makes the cut layer paradox-free: cut is a closed-form function of the order sheets plus convoy viability – no fixpoint among supports, no cut-cascade ordering. The rulebook mandates the **sum** of all qualifying attacks ("diminished by the combined Strength of all Support-cutting Attacks"), not the strongest one. $\mathrm{eff}(s)$ is floored at 0.

Degenerate reading (standard): with base $\equiv 1$, every qualifying cutter a has $\mathrm{cutval}(a) \geq 1 = \mathrm{eff}_0(s)$, so $\mathrm{eff}(s) = 0$ iff $\mathrm{QC}(s) \neq \emptyset$ – the classic binary rule, exception included (Lemma L2).

3.3 Dislodgement voids; role filters

- **Dislodge-void.** A support (or convoy) given by a unit that is *dislodged* delivers nothing to its subject: $\mathrm{delivered}(s) = 0$ (Sec. 2; classic "support is cut if the supporting unit is dislodged", which applies even when the dislodging attack came from the protected province; Beleriand inherits it through the rulebook's "Except for the

rules listed in this rulebook, the standard Diplomacy game rules apply"). Voiding is a property of the final labeling (Sec. 5), *not* of the one-shot cut pass: eff_0 and cutval never read it, so a dislodged unit's pre-void strength still counts inside cutval, by the definition of cutval (Sec. 3.2). At base $\equiv 1$ this makes no difference, since every classic cut is delivered by a Move and not by a support, and a doomed unit's Move still cuts.

The exception (Beleriand; the standard instantiation has no Bar): a support that backs a Bar is not voided – $\text{delivered}(s) = \text{eff}(s)$ whatever becomes of $\text{unit}(s)$ (Sec. 2). The rulebook: "A Support given to a Barring Order is cut as any other Support, but it is not lost when the Unit giving it is Dislodged in the same Phase: in the same moment in which that Unit is Dislodged, it Supports the Bar, so both happen" – the reason it gives for the Bar itself, which takes effect even if the barring unit is dislodged (Sec. 4.2), applied to the units that back it. The unit is dislodged all the same; only its support of the Bar outlives it. Cutting is untouched: such a support is cut by every qualifying cutter, and the unit its Bar is directed against is still not one of them (Sec. 3.1, condition 3). The consequence is that no dislodgement voids a support of a Bar, neither the barring unit's nor that of a unit backing it (Sec. 4.2): a Dislodge cannot weaken the Bar that gates it by dislodging one of the Bar's backers.

A dislodgement still reaches the strength of a Bar in one way, through a convoy: dislodging a convoying fleet can decide whether a convoyed attacker exists (Sec. 3.1, condition 4), and so whether it cuts a backer of the Bar.

- **Role filters** (standard, inherited by Beleriand): in the contest role attack against a defender d, the supports given by $\text{pow}(d)$ are left out of the sum (you cannot help dislodge your own unit); in the contest roles prevent and defend all matching supports count. Sec. 4 writes this filter into the definition of attack. A move by $\text{pow}(d)$ can never dislodge d at all (Sec. 4.2), though its prevent strength stands.

3.4 Cut-target selection in Areas (Beleriand)

A Dislodge names a victim by a unit reference – owner, kind, base strength, designator and province – and optionally the victim's exact order. It is the only attack on a unit standing in an Area: a Move into an Area names nobody and cuts nothing there (the rulebook: "A Unit that simply Moves into an Area does not Attack the Units standing in it; only a Dislodge Order Attacks a Unit in an Area"; Sec. 3.1, condition 1). Say that a unit *has* a named order iff its own order is of the named type and acts where the named one does: a Move into the named destination province, whatever coasts are written; a Hold; a Dislodge, or a Bar into the same Area, whose victim reference agrees with the named one in owner, kind, base strength, designator and province; a Convoy of an army from the named province to the named destination province, whatever kind, base strength and coasts are written; a support of an order of a unit standing in the named subject's province, whatever kind and base strength are written for that unit, that order having the named subject order by these same rules – the rulebook's `A(Est) dis Morgoth's A(NEB) sup A(Gel)-Tha` names the supported army by Gelion Headwaters. If the attack names the victim's order and **no unit has that order, no support is cut and the attack fails** outright. When the named order is a support, the attack cuts exactly that support; a named non-support action cuts nothing. An attack whose reference describes

one unit takes that unit as its victim and cuts that unit's support, if its order is one: the classic victim-coupled cut, which the standard map has at every province, where a Move cuts the one support standing in its destination. An attack whose reference describes no unit at all, or whose named order no unit that its reference describes has, is *mis-specified*: it fails, cuts nothing and does not exist as an attacker (Sec. 3.1, condition 4; Sec. 4.2); the order stays valid, since whether a reference resolves is decided here and not in Phase 0.

3.5 Convoy viability

A convoyed Move $m \in \mathcal{M}$ is *viable* iff it has an ordered chain $(q_1, \ldots, q_k)$ (Sec. 2) in which every q_i holds a fleet whose order is a Convoy of m and that is **not dislodged** (DATC 4.A.1 choice B: disruption requires all routes disrupted; 4.A.6 choice A: no route specifications – any qualifying chain counts). Beleriand adds a capability gate: convoying a Double Army requires Double Fleets – a fleet f may carry an army a only if $\text{base}(f) \geq \text{base}(a)$ – and a Convoy whose fleet may not carry the army that its cargo spec describes is illegal (Sec. 1): it is ignored, and its fleet takes its next legal order or holds, so every fleet of a chain satisfies the gate. The gate has a second clause, which the base strengths do not decide: a fleet f may carry an army a of MORGOTH only if $\text{pow}(f) = \text{pow}(a)$, so no fleet of another power carries a unit of MORGOTH (the rulebook: "No Morgoth's Unit may be on the Sea or Convoyed by other Powers"). A Move written without `via convoy` whose army reaches its destination is overland even if fleets offer to convoy it; a Move to a coastal province that its army does not reach becomes a convoy attempt without the words iff the fleets standing on the board could carry the army there (Sec. 1), and it is illegal otherwise (4.A.3 as pinned in the Beleriand rulebook; the standard ruleset makes the same choice, which departs from the v3.3 document's preferred outcome in six cases – 6.G.1, 6.G.5, 6.G.6, 6.G.9, 6.G.11 and 6.G.14 – listed in `docs/datc-choices.md`).

4 Contests

The rules of a movement phase are stated as a set of interlocking *contests*: a contest compares strengths and decides one local question – who wins an edge, who is displaced, who enters a province. *Adjudication* itself is the labeling of every relocating order as succeeding or failing that settles all the contests at once, the verdict on every stationary order being derived from it; I define both in Sec. 5.1. Each contest consumes pipeline values (Sec. 2) in four *contest roles*. The standard instantiation's roles coincide with the DATC document's own vocabulary (hold, attack, defend and prevent strength), which I adopt for traceability and generalize. Each contest role is a function into the nonnegative rationals; $u, d \in \mathcal{U}$ are units and $m \in \mathcal{M}$ is a relocating order:

Role	Type	The strength with which …	Used in
hold(u)	$\mathcal{U} \to \mathbb{Q}_{\geq 0}$	u resists being displaced from the province it stands in	displacement (Sec. 4.2)
attack(m, d)	$\mathcal{M} \times \mathcal{U} \to \mathbb{Q}_{\geq 0}$	m tries to displace the defender d, a unit standing in dest(m)	displacement (Sec. 4.2)
defend(m)	$\mathcal{M} \to \mathbb{Q}_{\geq 0}$	m resists the opposite move in a head-to-head	edge contest (Sec. 4.1)
prevent(m)	$\mathcal{M} \to \mathbb{Q}_{\geq 0}$	m keeps its rivals out of dest(m)	entry (Sec. 4.3)

For every relocating order $m \in \mathcal{M}$, define

$$\text{prevent}(m) \coloneqq \text{value}\Big(\text{mods}(\mathsf{move}, m),\ \text{supp}(m)\Big),$$
$$\text{defend}(m) \coloneqq \text{value}\Big(\text{mods}(\mathsf{move}, m),\ \text{supp}(m)\Big),$$

and for every relocating order $m \in \mathcal{M}$ and every unit $d \in \mathcal{U}$, define

$$\text{attack}(m, d) \coloneqq \text{value}\Big(\text{mods}(\mathsf{move}, m),\ \{s \in \text{supp}(m) \mid \text{pow}(\text{unit}(s)) \neq \text{pow}(d)\}\Big).$$

All three start from the modified base of the relocating order. prevent(m) and defend(m) add what every matching support delivers; attack(m, d) leaves out the supports given by the defender's own power (the role filter of Sec. 3.3), so it depends on d only through pow(d), and a contest consumes it only for a defender d standing in dest(m).
defend(m) is the strength of a relocating order that is itself being attacked along its own edge. When m meets an opposite m', with dest(m) = prov(m') and dest(m') = prov(m), the unit of m is leaving prov(m), so it does not answer m' with hold: it answers with the full strength of its own relocating order, every matching support counting. As numbers, defend(m) = prevent(m). They are two roles because two different contests consume them – defend the edge contest of Sec. 4.1, prevent the entry rivalry of Sec. 4.3 – and because only prevent can be nulled (prevent-nulling, Sec. 4.2).

For every unit $u \in \mathcal{U}$, with o_u the order it received, define

$$\text{hold}(u) \coloneqq \begin{cases} \text{value}\Big(\text{mods}(\mathsf{hold}, o_u),\ \text{hsupp}(u)\Big) & \text{if } o_u \text{ is stationary,} \\ \text{base}(u) & \text{if } o_u \text{ is relocating.} \end{cases}$$

A stationary unit resists with its modified base plus what its support-holds deliver: the sum inside value runs over $s \in \text{hsupp}(u)$. In the modified base mods(hold, o_u) the argument hold is the strength role of Sec. 2, not this function, and the fortress bonus enters there. A unit ordered to relocate is asked for its hold only when that order failed and it remains; it then defends with its bare base alone: a support-hold written for it matches nothing and becomes a Hold (Sec. 1), no modifier applies, and the fortress bonus is forfeited. Against an order that it met head-to-head (Sec. 4.1) it resists with defend of its own order instead of its hold (Sec. 4.2).

4.1 Edge (head-to-head) contests

Say that a province $p \in P$ is *displacement-governed* iff cap(p) = 1: the one unit that may stand in it is displaced by a successful entry (the displacement policy of Sec. 4.3).
Say that an *edge contest* exists between two relocating orders $m, m' \in \mathcal{M}$ iff neither of them is convoyed and one of the following two cases holds:

- m and m' are Moves with $\text{dest}(m) = \text{prov}(m')$ and $\text{dest}(m') = \text{prov}(m)$, and **both provinces are displacement-governed** – two units cannot swap places overland.
- (Beleriand) m is a Dislodge and m' is the relocating order of its victim (Sec. 3.4), or, where m is an unnamed Dislodge against a stack of twins, of any twin of that stack (Sec. 4.4, rule 6) – a Move, or a Dislodge of a unit standing in $\text{prov}(m)$ – with $\text{dest}(m') = \text{prov}(m)$, whatever the capacities of the two provinces. The victim cannot slip past the unit that attacks it (the rulebook: "If the Unit to be Dislodged is ordered to Move into, or to Dislodge a Unit in, the Province of the Dislodging Unit, the two Orders clash head-to-head, as two opposite Moves do").

The winner is the order with the strictly greater defend value, and the loser's order fails. Winning does not by itself admit the winner: it still has to complete its entry (Secs. 4.2 and 4.3), and otherwise it fails as well. A Move into a province of capacity 1 that completes its entry dislodges the loser in place, the loser resisting it with defend of its own order (Sec. 4.2). A Move into an Area displaces nobody (Sec. 4.2), and neither does a Dislodge whose victim has left: the loser stays. A Dislodge that wins its displacement contest (Sec. 4.2) dislodges its own victim, which is the loser's unit unless the winner is the victim's order m' of the second case and names another unit standing in $\text{prov}(m)$; that unit is then dislodged, and the loser stays. Ties fail both. A convoyed Move fights no edge contest: convoyed swaps are legal (DATC 6.G behavior). In Beleriand, an Area is not displacement-governed, so cross-Area "swaps" – two opposite plain Moves of which at least one enters an Area – are simply two independent entries, and the two units pass each other (BATC.B.7 checks it between two Areas); only a Dislodge meets the relocating order of its own victim. Besides the relocating order of its victim, a Dislodge m meets head-to-head only a Dislodge m'' with $\text{prov}(m'') = \text{dest}(m)$ whose victim is $\text{unit}(m)$ – the second case above with the roles of the two orders exchanged (BATC.C.19) – and, where the relocating order of its victim enters $\text{prov}(m)$ as a member of a wave (defined in Sec. 4.4), every member of that wave (Sec. 4.4, rule 2). Every other unit that leaves the Area of m for $\text{prov}(m)$ fights no edge contest with m and passes it, as two plain Moves across the border of an Area do; its entry into $\text{prov}(m)$ is an entry like any other (Sec. 4.3), so at capacity 1 it enters only where the unit of m leaves or is displaced by it (Sec. 4.2). The reason is the size of an Area: two units moving through one miss each other unless one of them hunts the other, which only a Dislodge does (the rulebook: "only a Dislodge Order Attacks a Unit in an Area"); and a Dislodge that had to beat every unit leaving the Area towards it, besides the Bars and the rival Dislodges against it and its victim's defence, would hardly ever succeed.

4.2 Displacement contests

Say that a stationary (or failed-relocating) unit d at a province p of capacity 1 is *dislodged* iff some relocating order m with $\text{dest}(m) = p$ **succeeds** while d remains: m is the admitted entrant (Sec. 4.3), $\text{pow}(\text{unit}(m)) \neq \text{pow}(d)$, and $\text{attack}(m, d) > \text{hold}(d)$, with $\text{defend}(o_d)$ in place of $\text{hold}(d)$ where the order o_d of d met m head-to-head (Sec. 4.1). In an Area entry displaces nobody, and a unit d standing there is displaced only by a Dislodge whose victim is d, through the reduction below. Only the admitted entrant dislodges, so an attack that bounces on a rival dislodges nobody: in the beleaguered garrison two supported attacks of equal strength both bounce and the defender survives (DATC 6.E behavior). In a

head-to-head the defender resists with defend, not hold (Sec. 4.1).

Beleriand Areas – **entry displaces nobody**. The only displacement contest in an Area is a targeted **Dislodge** order, which I formulate as the *pseudo-province reduction*: each targeted unit w is, for adjudication only, a virtual province p_w of capacity 1 occupied by w. Dislodge orders naming w are moves into p_w, their supports are supports into p_w, and the whole Secs. 4.2 and 4.3 machinery applies verbatim. In particular, the strongest dislodger has to strictly beat its rivals, every other Dislodge of w but one that lost or tied an edge contest with the order of w (below), and beat hold(w): several Dislodges against one victim can stand each other off. The reduction *derives* the rulebook's per-unit cut exception from the standard per-province one: an attack "from p_w" is an attack by w itself, so a support directed against w is protected exactly from w. Pseudo-provinces are an adjudication-time construct – they never appear in map data, state, or serialization. A Dislodge that dislodges its victim enters the real Area in the slot that its victim frees, before plain moves are admitted (Sec. 4.3; BATC.C.14). A mis-specified Dislodge – its reference describes no unit, or no unit that it describes has the named order (Sec. 3.4) – fails (BATC.C.9 and BATC.K.15). A Dislodge whose victim does not remain – the victim's own relocating order succeeds – and that fights no edge contest with that order meets no displacement contest (the rulebook: "If the Unit to be Dislodged leaves the Area without clashing with the Dislodge, the Dislodge becomes a Move into the Area and competes for its space as a Move does"): it is a plain candidate for the Area in the entry rule of Sec. 4.3, as a Move into the Area is, prevent-nulling (below) included. A Dislodge m that fights an edge contest with its victim's relocating order m' (Sec. 4.1) and wins it finds its victim w remaining, and its displacement contest is the one above with defend(m') in place of hold(w): m displaces w iff no Bar stops m (below), m wins every other edge contest it fights, and attack(m, w) is strictly greater than defend(m') and than the prevent of every other Dislodge whose victim is w. The victim then resists with the strength of its order, as a head-to-head defender does. A Dislodge that loses that edge contest, or ties it, has failed (the rulebook: "A Dislodge that loses its clash fails, and it does not stand in the way of the other Units entering the Area"). Whether its victim then leaves or remains, the edge contest is the only contest of this section and of Sec. 4.3 that it takes part in: it is no candidate for the Area (Sec. 4.3), its prevent stands off no other Dislodge of its victim, and it makes no province embattled (Sec. 9; BATC.B.24 to B.27 and C.21).

Bar contests (Beleriand). Say that a Bar b *binds* the entrant its victim reference describes: the relocating order, a plain Move **or a Dislodge**, of the unit it names whose destination is the Area of b (the rulebook: "ordered to Bar against a Unit which attempts to enter that Area – by a Move Order or by a Dislodge Order"). The Bar guards its door against the *unit* it names, whatever that unit's entry order says. A Bar whose victim received no such order binds nothing. Where its reference fits several twins, it binds the entry of one of them only (Sec. 4.4, rule 3).

An entry m into an Area is stopped by a Bar b bound to it iff $\text{prevent}(m) \leq \text{value}\Big(\text{base}(\text{unit}(b)),\ \text{supp}(b)\Big)$. So the *Bar wins ties* (the rulebook says so in words: "a tie favours the Barring Order"). The right-hand side is the Bar's strength: its unit's *unmodified* base plus what the supports of the Bar deliver. A Bar has no strength role and takes no modifier (the rulebook: "Barring Orders are never halved"). Every $s \in \text{supp}(b)$

backs a Bar, so it delivers its effectiveness (Sec. 2), and the Bar's strength is

$$\mathrm{value}\Big(\mathrm{base}(\mathrm{unit}(b)),\ \mathrm{supp}(b)\Big) = \mathrm{base}(\mathrm{unit}(b)) + \sum_{s \in \mathrm{supp}(b)} \mathrm{eff}(s).$$

It reads no dislodgement: like cut (Sec. 3.2), it is a closed-form function of the order sheets and the convoy viabilities. A barred Dislodge fails wholly even when it beats the victim's defence: the victim stands, and the dislodger stays home. A second Bar of the same power bound to the same entrant does nothing additional: of several, the Bar of the strongest barring unit stands (the first written, if they are equal), every other becomes a Hold, and their supports attach to the standing Bar (the rulebook: "Only one Barring Order of a Power stands against the same barred Order: the strongest of them stands (the first written, if they are equal), every other does nothing additional and is changed into a Hold Order"; its twins rule 3, which holds against every barred order alike). The Bars of different powers bound to one entrant stand separately, each tested on its own. Bar strength is supportable. A Bar order is stationary and **cannot be cut: it takes effect even if the barring unit is dislodged in the same phase** (twins rule 4: "all Orders happen at the same time"; BATC.D.5). Reading the dislodgement as voiding the Bar would contradict that simultaneity. The barring unit defends as a holding unit. A barred entry keeps exerting its prevent, like any bounced move (BATC.D.6). The supports of a Bar *can* be cut by every qualifying cutter, and the unit the Bar is directed against is not a qualifying cutter of them (Sec. 3.1, condition 3). They are **not lost by dislodgement** (Sec. 3.3). So an attack on a backer by any other unit weakens the Bar by what it cuts (BATC.D.12 and BATC.D.13), while the entrant that the Bar is bound to can neither cut the Bar's supports (BATC.D.10) nor take them away by dislodging their givers (BATC.D.11). A Dislodge that a Bar stops has attacked its victim all the same: it is a qualifying cutter of that victim's support whether or not it gets in, since a Dislodge is overland and whether an overland attacker exists reads no verdict (Sec. 3.1, condition 4; BATC.D.14).

Prevent-nulling (standard, inherited). A unit dislodged by an attack *from its own destination* exerts prevent = 0 there (a dislodged unit has no effect on its dislodger's province – DATC 6.E behavior), **unless** the dislodger arrived by convoy (DATC 4.A.7 choice B: a unit dislodged by a convoyed attack still affects the origin of the attacker).

4.3 Entry (capacity and admission) – the province contest

For each province $p \in P$, let the *candidates* be the relocating orders m with $\mathrm{dest}(m) = p$ that exist as attackers (Sec. 3.1, condition 4) and, for a Dislodge, have a victim that does not remain and have not lost or tied an edge contest with their victim's order (Sec. 4.1). A Dislodge whose victim remains contests the victim's slot instead (Sec. 4.2). A Dislodge that lost or tied its edge contest with its victim's order contests neither the Area nor its victim's slot, whether its victim has left or remains (Sec. 4.2). The victim of an unnamed Dislodge against a stack of twins is the twin that the engagement gives it, and it has none where the engagement gives it none: where it finds no twin left, or where its cohort or a stronger one stands off (all defined in Sec. 4.4, the engagement in its rule 6). Such a Dislodge is no candidate: it competes for no slot, and its prevent keeps no other candidate out, though it still exists as an attacker, cuts (Sec. 4.4) and counts toward an embattled province (Sec. 9).

Let the *admission rank* of a candidate m be prevent(m); Sec. 4.4 lowers it for the members of a wave that are not its champion. A candidate m whose prevent is nulled at p (Sec. 4.2) is no rival: the entry of every other candidate is decided as if m were absent, while m still competes for its own entry at its full admission rank. The edge contest (Sec. 4.1) and the Bar contests (Sec. 4.2) decide only whether a candidate may itself be admitted: a candidate that loses them keeps its place in its class of the ladder (below) at its full admission rank and can still bounce that class – a barred or edge-beaten relocating order keeps exerting its prevent, like any bounced one, a barred Dislodge against its rivals in its victim's pseudo-province included. The one exception is a Dislodge beaten or tied in its edge contest with its victim's order, which is no candidate at all (above). Entry permission is decided in Phase 0 (Sec. 1): an order it refuses is illegal and ignored, so it is no candidate, and its unit takes its next legal order or holds (Sec. 6).
Let the *stayers* be the occupants of p that remain: the units with a stationary order, and the units whose relocating order fails. Stayers occupy slots first, and every other occupant vacates its slot; a stayer that is dislodged keeps no slot either, but its slot goes to its dislodger (Sec. 4.2) and to no other candidate.

Admission rule (ruleset data), the *ladder*. The candidates are sorted into classes by descending admission rank. Candidates are admitted class by class while the whole class fits within cap(p) − #(stayers) − #(candidates already admitted), every stayer counting whether or not it is dislodged (above), and **the first class that does not wholly fit bounces, and admission stops** (BATC.B.4). No weaker class squeezes past a standoff. The reason is that the rulebook's greedy-descending procedure is stated for the all-admitted prefix, and stopping is the reading consistent with the classic rule that a standoff blocks all weaker entrants.

Displacement policy (ruleset data, a predicate of cap(p)). In a province of capacity 1 – every province of the standard map, and every Beleriand province that is no Area – the single top candidate may free the defender's slot by winning the displacement contest (Sec. 4.2). In an Area no entry frees a slot: only a targeted Dislodge does. At capacity 1 the ladder takes the following form, the displacement policy included. Let d be the stayer that remains in p, if any. A candidate m enters iff it wins every edge contest it fights (Sec. 4.1), attack(m, d) – prevent(m) where no stayer remains – is strictly greater than the admission rank of every other candidate whose prevent is not nulled at p (Sec. 4.2) and, where d remains, m also wins the displacement contest of Sec. 4.2. One tie at the top bounces everyone – **exactly the classic bounce rule** (Lemma L3). The candidate is measured by its attack on the stayer, not by its prevent, so a support given by the defender's own power never carries an attacker past a rival either (DATC 6.E.7). Off-map boxes (capacity 1) behave classically on the Beleriand map with no special-casing – an internal degeneracy check on one map.

Vacancy chains. Whether an occupant vacates its slot depends on the entry contest of its own relocating order elsewhere. Chains of such dependencies, and cycles of them (three units rotating), are the first circularity source (Sec. 5.2). A failed candidate remains a stayer at its origin and may be dislodged there.

4.4 Twins (Beleriand)

Where units that cannot be told apart stand together in an Area the rules above bend for them, and this subsection holds every rule that exists for that reason; the rulebook's section "Indistinguishable Units (twins)" is its normative text. The rules below keep the numbers of that section; its rule 4 – a Bar takes effect even if the barring unit is dislodged – is not about twins and stands in Sec. 4.2. Nothing here changes anything for a unit that is nobody's twin – the rules below bend only for the members of a stack (both defined below), whatever their orders say – and nothing here reaches the standard map, where no province holds two units.

Twins. Let the *signature* of a unit be the designator its orders write (Sec. 1): `(M)` for the Morgoth unit, `(G)` for Glaurung, and nothing for every other unit.
Say that two different units $u, u' \in \mathcal{U}$ are *twins* iff they agree in power, kind, base strength and signature and stand in one Area: $\mathrm{prov}(\mathrm{loc}(u)) = \mathrm{prov}(\mathrm{loc}(u'))$ and $\mathrm{cap}(\mathrm{prov}(\mathrm{loc}(u))) > 1$.
Let a *stack* be a set of at least two units that are pairwise twins and to which no further twin of theirs can be added, its members listed in the order of their power's sheet. A member stands in that list at the place of the first legal order that Phase 0 binds to it, or of the first order bound to it where none is legal (Sec. 6); the members to which no order is bound come after all the others, by canonical index (Sec. 1).
The Morgoth unit and Glaurung agree in power, kind and base strength and are never twins, since their signatures differ (the rulebook: "Morgoth and Glaurung are therefore never twins"): they join no stack, and so form no wave (rule 2) and pool no wall (rule 5) together. The signature is what the engine keys a twin class on, and a special carries it whatever its order says (Sec. 1); which special is which is recorded in the game layer's unit ledger (Sec. 1).
Twins that received identically worded orders are *indistinguishable*: no written specification can tell them apart. An order or a reference is neither illegal nor invalid for fitting several twins, whether or not their orders agree (the rulebook's 4.C.5 reading): it takes effect by the rules below, and no player makes any choice during resolution. The ambiguous unit reference that Sec. 1 calls illegal is one that fits two units that are not twins of each other. A twin whose order differs from its twins' can still be named by that order (a Dislodge naming the victim's action, Sec. 3.4).
Sec. 1 binds a support, a Bar and a Dislodge to "the unit its reference describes"; where the reference describes several twins, the rules below say which one, and they read the order of the sheets: this is why sheets are ordered lists and why `power_symmetric = False` for Beleriand (Sec. 7).

Rule 1 – supports concentrate; the champion. Let $o_1, \ldots, o_n$, $n \geq 2$, be identically worded orders of the members of one stack. Every support whose subject reference fits one of these units and whose subject order is their common order matches each of $o_1, \ldots, o_n$ alike (Sec. 1); it is bound to one of them, their *champion*: the order among them that the sheet of the stack's power lists first. The engine binds it to the one among them of least canonical index (Sec. 1), which is that same order, since Phase 0 binds each order of a sheet to the twin of least canonical index that no earlier legal order has bound (Sec. 6). Where the common order is relocating or a Bar, Sec. 1's supp is read so: supp of the champion holds every support that matches the common order and is no support-hold, and $\mathrm{supp}(o_i) = \emptyset$ for every other o_i – "it is not possible to divide such Supports among

the twins in any other way"; where it is stationary, rule 5 says what the support-holds given to the stack do. Where twins' orders differ, a support binds a twin that received its subject order, whichever twin the sheet lists first: with the sheet `A ANF H`, `A ANF - LOT`, `A ANF S A ANF - LOT` the support backs the mover and never the holder written above it (BATC.B.15/B.16).

Rule 2 – waves. Let a *wave* be a set of at least two identically worded overland Moves of the members of one stack; its champion is the champion of rule 1. Every member m of a wave shares the champion's values and fate through the gates: prevent(m), defend(m) and attack(m, d) are those of the champion, so the wave's strength is one unit plus all the supports (rule 1) and its size adds no strength, and a Bar bound to the champion stops every member (rule 3). Only admission tells the members apart: the admission rank (Sec. 4.3) of a member that is not the champion is its own modified base mods(move, m), not its prevent. So a wave enters the ladder of Sec. 4.3 member by member: a supported champion strictly outranks its twins and takes a too-small door alone, while an unsupported wave is one equal class and bounces whole where it does not fit (the rulebook: "where the whole wave does not fit, a Supported strongest Move may enter alone, and Moves of equal Strength which do not all fit, all fail"). At a door of capacity 1 this is the classic picture of a supported Move among unsupported rivals. Therefore the wave enters as one only when it fits the free space. Gates are prior to admission and stay wave-wide: a standing Bar stops every member. So is the edge contest of Sec. 4.1: a Dislodge whose victim is a member of a wave, or an unnamed Dislodge against the wave's stack, and whose unit stands in the wave's destination meets the whole wave, at the champion's defend, and where the Dislodge wins or ties, every member fails, the wave losing as one (BATC.J.8, J.9), while a wave that wins enters member by member as above (BATC.J.10).

Rule 3 – one Bar gates the whole wave. Where a Bar's victim reference fits several entrants of a stack, the Bar binds the first of them in the victim power's sheet and no other (Sec. 4.2); when that entrant is a member of a wave it is the wave's champion, so **one Bar gates the whole wave**: the wave passes or fails together, and against the Bars of several powers it has to pass each. No other entrant of the stack is barred by it, unless it is a member of the bound entrant's wave: where twins enter by a Move and by a Dislodge, or by two Dislodges, the sheet alone decides which entry the Bar is tested against. The rule on a second Bar of one power against the same bound entrant, and on the Bars of different powers, is that of Sec. 4.2 and holds against a wave as against a lone entrant.

Rule 5 – the wall. For a member u of a stack, let its *wall* wall(u) $\subseteq \mathcal{S}$ be the set of support-holds that match the order of some member of the stack and are not given by u itself: every support-hold whose subject reference fits the stack and whose subject order is stationary, whichever member each is bound to, less the one u gives, if any. For a member u whose order o_u is stationary, the wall takes the place of hsupp(u) in Sec. 4:

$$\mathrm{hold}(u) \coloneqq \mathrm{value}\Big(\mathrm{mods}(\mathsf{hold}, o_u),\ \mathrm{wall}(u)\Big).$$

The rulebook: "The defence of every twin in the stack against Dislodgment is its own Strength plus ALL such Support Holds – except the Support Holds that twin itself gives"; six mutual supporters defend at 6 each, not 7. A member ordered to relocate keeps its bare base (Sec. 4); a member ordered to Bar defends as a holding unit with the wall behind it.

Rule 6 – the engagement of unnamed Dislodges. Call a Dislodge *unnamed* iff it names no victim order.

Let a *walk group* be a stack that the victim reference of at least one unnamed Dislodge fits, together with every unnamed Dislodge whose victim reference fits its members. A stack that no unnamed Dislodge attacks forms no walk group. An attack belongs to one walk group at most, since the units one reference fits agree in power, kind, base strength, signature and province (Sec. 1) and so are twins of one another.

Let the *pool* of a walk group be the members of the stack in the list order of the attacked power.

A support of an unnamed Dislodge against a stack is directed against nobody (Sec. 1): the attack's victim is decided only by the engagement below. Stack Dislodges engage **strongest-first over the victim's list order**: the unnamed Dislodges of a walk group that exist as attackers (Sec. 3.1, condition 4) are sorted by attack against the attacked power, equal strengths forming a *cohort*. An unnamed Dislodge fights its edge contests before the engagement: it meets every twin of the pool whose relocating order enters its province (Sec. 4.1), whichever twin the engagement would give it. One that loses or ties such a contest joins no cohort and consumes no twin, and it is given the first twin of the pool that beats or ties it, for that edge contest alone (Sec. 4.2; BATC.J.11 and J.12); one that wins every such contest is dealt a twin like any other attack. Attacker order sorts the attacks by their powers in canonical order, then by their sheet position. Each cohort in turn is dealt the twins of the pool that no stronger cohort has consumed (below), one twin per attack: the cohort's attacks in attacker order are paired with these twins in the pool's order. An attack *consumes* the twin it is given iff the attack succeeds or the twin has departed – its own relocating order succeeds. A twin that has departed keeps its place in the pool: the attack it is given is the vacated-victim entry of Sec. 4.2, and it consumes the twin whatever becomes of that entry. A twin that repels an attack is not consumed: it keeps standing for the next cohort. A failed attack is spent: it does not fall through to the next twin. A cohort larger than the twins not yet consumed stands off, failing itself and everything weaker. With one twin left that stays, this is exactly the classic lone-victim standoff; a twin that has departed counts among the twins not yet consumed as well, so a cohort can stand off where the Area has room. An attack that the engagement gives no twin – it finds no twin left, or its cohort or a stronger one stands off – fails while still cutting (below), and it is no candidate (Sec. 4.3). A named attack takes the first twin in the stack's list order that has the named order (Sec. 3.4); where several twins received that order alike, every attack that names it takes that same twin and attacks no other. Rival named attacks on one twin resolve as a lone-victim contest: the strictly strongest wins, and ties bounce. A twin that a successful named attack dislodges is out of the pool before the unnamed walk begins, while a twin that has departed stays in it whatever named attacks name it, each of them being the vacated-victim entry of Sec. 4.2. The engagement is a function of the labeling L (Sec. 5.1), like every contest of Sec. 4: whether a twin is consumed is read from L – the verdict on the attack it is given, and whether the twin remains – and not decided anew. So Eval of Sec. 5.1 evaluates a walk group under L without evaluating the same group again, since attack against the attacked power leaves out that power's supports (the role filter, Sec. 3.3); where another group's strengths, or the defend of the twins that the test for a beaten Dislodge reads, lead back to a group whose engagement is being evaluated, the engine reads that group as engaging nobody on that path. The resolver of Sec. 6 evaluates the engagement in its three-valued form in the same strength order and hands an engagement it cannot settle to the selection of

Sec. 5.1.

Cut-target selection on a stack (Sec. 3.4 for a single victim). The unnamed attacks of a walk group cut the stack's written supports *on their own list*: they cut the supports of the pool, first listed downward, one per attack in attacker order. The supports that the named attacks name come off that list first, whether or not those attacks cut them: a named attack that is the protected attacker (Sec. 3.1, condition 3) of the support it names cuts nothing, and it still takes that support off the list. A support is skipped when its protected attacker is the attack in hand. This cut-target selection is *decoupled* from the victim the engagement gives the attack (the rulebook: "The victim of an unnamed Attack and the Unit whose Support it cuts may therefore be different twins – this is intended").

Example (the rulebook's own; BATC.J.1 and its coda J.2). MORGOTH has four armies in ANF and orders `A ANF - LOT` twice and `A ANF S A ANF - LOT` twice; the FEANORIANS have four armies in LOT and order `A LOT BAR MORGOTH A ANF - LOT` twice and `A LOT S A LOT BAR MORGOTH A ANF - LOT` twice. Each support's subject reference fits both movers, and both supports attach to the champion, the first-listed Move (rule 1): the wave has strength $1 + 1 + 1 = 3$. The second Feanorian Bar is bound to the same entrant as the first and becomes a Hold (Sec. 4.2); both Feanorian supports attach to the standing Bar, so the gate has strength $1 + 1 + 1 = 3$; the tie favours the Bar, and neither Move enters (J.1). Give one of MORGOTH's supports to a `2A ANF` and the wave has $1 + 1 + 2 = 4 > 3$: both Moves enter, and LOT holds six units (J.2).

5 Semantics: stable labelings

5.1 Definition

Secs. 2–4 leave two facts open: which units are dislodged, and which convoys are viable. Both depend on which relocating orders succeed, and that depends on both. A labeling closes the circle by supposing an answer; it is a right answer when the rules give it back.

Labelings. Let a *labeling* be a function $L\colon \mathcal{M} \to \{\textsc{succeeds}, \textsc{fails}\}$: one verdict for every relocating order. The stationary orders are not labeled; their verdicts are derived from L, below.

What is read from a labeling. Let L be a labeling.
A unit $u \in \mathcal{U}$ *remains under* L iff o_u is stationary or $L(o_u) = \textsc{fails}$.
A unit $u \in \mathcal{U}$ is *dislodged under* L iff it remains under L and an order that displaces it (Sec. 4.2) has the verdict SUCCEEDS: a Dislodge whose victim is u, or, where displacement is implicit (capacity 1), a foreign Move whose destination is the province of u.
A convoyed Move $m \in \mathcal{M}$ is *viable under* L iff it is viable in the sense of Sec. 3.5 with "dislodged" read under L.
These are the inputs that Sec. 2 had to take as given: the dislodgements behind delivered and, through the existence condition of Sec. 3.1, the viabilities behind cut. With them every strength of Sec. 2 and every contest of Sec. 4 has a value *under* L; who vacates a province and whose prevent is nulled (Sec. 4.2) are read from L in the same way.

Eval **and stability.** Let Eval be the function from labelings to labelings defined, for

every labeling L and every $m \in \mathcal{M}$, by

$$\mathrm{Eval}(L)(m) := \begin{cases} \textsc{succeeds} & \text{if } m \text{ is admitted under } L, \\ \textsc{fails} & \text{otherwise,} \end{cases}$$

where m is *admitted under* L iff, with every fact read from L, it exists as an attacker (Sec. 3.1, condition 4), wins every edge contest it fights (Sec. 4.1), is stopped by no Bar (Sec. 4.2), and is a candidate that the entry rule of Sec. 4.3 admits – a candidate whose prevent is nulled there included, which only stops counting as a rival – while a Dislodge whose victim remains, or which meets its victim's order head-to-head, has to win its displacement contest instead (Sec. 4.2), and takes the victim's slot.
A labeling L is *stable* iff $L = \mathrm{Eval}(L)$.

The stationary orders. Let L be a labeling. Under L, a support $s \in \mathcal{S}$ *succeeds* iff it delivers, $\mathrm{delivered}(s) > 0$ – it is not fully cut and, unless it backs a Bar, its unit is not dislodged (for the standard degeneracy this is the binary cut status). A support that backs a Bar and is not fully cut therefore succeeds even when its unit is dislodged under L: it delivered (Sec. 2). A Convoy succeeds iff its fleet is not dislodged and the route of the Move it carries runs through it: that Move is viable and not put out of play by the Szykman rule (Sec. 3.1; Sec. 5.2), and chains of undislodged fleets convoying it link the fleet's sea to both coasts. A Hold succeeds iff its unit is not dislodged. A Bar succeeds iff it is bound to an entrant (Sec. 4.2) and that entrant's order fails – whatever becomes of the barring unit, since a Bar takes effect even when its unit is dislodged. A Bar bound to no entrant fails, and so does a second same-power Bar that Sec. 4.2 turns into a Hold. An order that Phase 0 replaced by a Hold is reported as failing, whatever that Hold does. So a labeling decides a verdict on every order in $\mathcal{O}$, and nothing is lost by labeling the relocating orders alone: the stable labelings correspond one to one to the stable verdicts on all of $\mathcal{O}$.

Components. Let D be the *order-dependency digraph*: a directed graph on $\mathcal{O}$ that Phase 1 builds from the syntax of the orders alone (Sec. 6). This subsection needs one property of it, Lemma L1: for every labeling L and every $m \in \mathcal{M}$, $\mathrm{Eval}(L)(m)$ depends on L only at the orders reachable from m in D.
Let a *component* be a strongly connected component (SCC) of D. The components form a directed acyclic graph; *dependency order* lists every component after all the components it reaches.

The selection. Let C be a component all of whose dependencies outside C are decided, and let the *residue* $R \subseteq C \cap \mathcal{M}$ be the set of its relocating orders that are not decided yet.
Let a *completion* of R be a function $c\colon R \to \{\textsc{succeeds}, \textsc{fails}\}$. Together with what is decided, a completion is a labeling at every order that R reads (Lemma L1), and it is *stable* iff Eval gives back $c(m)$ for every $m \in R$.
The *selection* settles R as follows.

1. If R has exactly one stable completion, that completion is taken.

2. If it has several, every $m \in R$ on which all of them agree is decided accordingly – a convoyed Move only if its viability is the same under all of them as well – and the selection starts again on what is left, stability still being required on every order of the original R.

3. If it has none, every $m \in R$ to which Eval gives the same verdict under every completion of R is decided with that verdict – the rules settle it on their own – and the selection starts again on what is left, stability still being required on every order of the original R.

4. If it has none and step 3 decides no order of R, or several from which step 2 decides no order of R, the residue is *genuinely ambiguous*: it is classified and settled by the ruleset's *backup policy* (Sec. 5.2).

Define *adjudication* as the labeling obtained by running the selection over the components in dependency order, together with the verdicts on the stationary orders that it decides.

The two engines decide different relocating orders before the selection, and only in the resolver are they most of $\mathcal{M}$. The enumerator decides beforehand only two forced facts: a convoyed Move for which no chain was ordered fails, and an overland Move to which no Bar is bound, into a province where no unit stands and at which no other relocating order is aimed, succeeds. The resolver decides everything that its propagation forces, and the cycles that its vacancy commit V settles (Sec. 6). T3 is the claim that the difference cannot be observed on any order from which no backup-resolved component (defined in Sec. 8.3) can be reached. Where C has no stable completion, the two engines may hand the selection different residues. Step 3 decides in both what the rules settle on their own before the backup policy runs. That the two then agree is checked by outcome (`resolver == enumerator`), not proved.
The adjudication yields `PhaseResult`: per-order statuses, the state after the phase, the dislodgements with what the retreats need to know of each dislodger, and the embattled provinces (Sec. 9). The statuses are reported per written order: every order written on a sheet has one row, which carries the order in the notation of Sec. 1, and the rows follow the powers in canonical order (Sec. 1) and each power's orders in the order of its sheet. An order that Phase 0 ignores or passes over (Sec. 6) – an illegal one, one written for a unit that an earlier legal order binds, or one that binds no unit of its power – is reported as failing. The rows of identically written orders of one power report on the units that those orders bind, in sheet order: the k-th such row reports on the k-th such unit, and a row with no unit left for it fails. A unit for which no order was written has no row. The state after the phase holds every unit that is not dislodged at its location after the phase, within the capacity bound of Sec. 1, and its phase is that of the state read, which the adjudication does not advance. A dislodged unit is not in it: it appears only among the dislodgements, which the retreats read (Sec. 9).

5.2 The two circularity sources

A residue is genuinely ambiguous (Sec. 5.1) only when the rules run in a circle, and in the standard instantiation they do so in two ways. That these two are all the ways is argued, not mechanically checked (Sec. 8.4).

1. **Move/vacancy cycles** are formed by units chasing each other's provinces in a cycle. Say that such a cycle is *clean* iff its Moves are overland, no support matches any of them, and no relocating order outside the cycle is aimed at any of its provinces. In a clean cycle of at least three Moves both all-succeed and all-fail are stable. For example: in a clean 3-rotation each Move succeeds iff its destination vacates iff

the next Move succeeds. The policy is **all succeed**, the classic circular-movement rule (DATC 6.C: three-army rotation succeeds). A cycle "disrupted" by an external bounce or a lost head-to-head never reaches the policy: the beaten member fails in every stable completion, so it is decided before the policy is asked. The beaten member is decided by step 1 or 2 of the selection, or by the resolver's propagation, and the rest unravel acyclically (DATC 6.C disrupted-rotation behavior).

2. **The entanglement of a convoy, a support cut and a dislodgement** is that of a convoyed attack whose landing cuts a support that feeds the sea battle deciding that same convoy's viability. Depending on the sign of the coupling, this yields two stable completions or **zero**. It yields two where the convoyed cut protects its own convoy: this is the self-protecting paradox (DATC 6.F.14). It yields none where the convoyed cut would disrupt its own convoy: this is Pandin's paradox (DATC 6.F.16). The policy is **Szykman** (DATC 4.A.2 choice F, chosen in both rulesets).

Classification. Let R be a genuinely ambiguous residue of the component C. Let the completions *considered* be the stable completions of R if there are several, and all the completions of R if there is none.
The viability of a convoyed Move $m \in C \cap \mathcal{M}$ *varies* iff m is viable under one of the completions considered and not viable under another.
The residue R is *convoy-entangled* iff the viability of some convoyed Move of C varies; otherwise it is a *pure cycle*. A rotation that merely *contains* a convoyed Move whose viability does not vary is a pure cycle and rotates successfully (DATC 6.C convoyed-rotation behavior).

The two policies. Let R be a genuinely ambiguous residue.
All succeed is the policy of a pure cycle: every order of R is given the verdict SUCCEEDS. Then every order of R that Eval refutes under this tentative labeling is turned to FAILS, all of them at once, each read against the same tentative labeling. That step is repeated until Eval refutes none. Nothing is ever turned back, so the process ends, and every SUCCEEDS that it leaves is one that Eval confirms under the final labeling. A clean rotation loses nobody.
Szykman is the policy of a convoy-entangled residue: every convoyed Move whose viability varies fails and is treated as holding. It does not exist as an attacker, so the cuts it would have delivered do not happen and it prevents nothing; its army holds, and the fleets ordered to convoy it count as holding too. The fleets may still be attacked and dislodged. The selection then starts again on what is left of R.

The one-shot cut arithmetic of Sec. 3, the rulebook's rule generalized to both instantiations, removes *support-vs-support* circularity by construction, and Beleriand's Area entry regime removes most vacancy coupling. An Area with room for every candidate even if all its occupants stay admits them whether or not anyone leaves (Sec. 4.3). The claim that the two sources above are complete is made for the standard instantiation only. In Beleriand a residue with no convoyed Move can be genuinely ambiguous without being a rotation, and it can have **no** stable completion at all: the position W of Sec. 8.4 has none. Its circle runs through three rules of Sec. 4: a Dislodge whose victim leaves collapses into a plain entry (Sec. 4.2), an entry into a province of capacity 1 waits on its occupant's departure where it cannot displace that occupant (Sec. 4.3), and a full Area admits by the ladder of Sec. 4.3. In W the victim moves into a province whose own unit moves into the dislodger's province, so it leaves iff the Dislodge succeeds, while the slot it leaves goes to a

stronger plain entrant, so the Dislodge succeeds iff the victim stays. A victim that moves into the dislodger's own province meets the Dislodge head-to-head instead (Sec. 4.1), and that contest decides it.
A further candidate source is *dislodge-void feedback*: a Dislodge whose success voids a support that feeds the contest deciding that same Dislodge. The definition of delivered closes the one door that is known for it: the victim of a Dislodge backing the Bar that gates that Dislodge. A support that backs a Bar is not lost with its giver (Sec. 3.3), so no dislodgement voids a support of a Bar, and a dislodgement reaches the strength of a Bar only through the existence of a convoyed cutter (Sec. 3.1, condition 4). Lemma L4 bounds what is left: where no walk group (Sec. 4.4) exists, a support that survives its cut is voided by nothing but its protected attacker. Whether a residue of this kind exists is not known; the search reported in Sec. 8.4 met none, and met no position without a stable completion either.
A residue whose component holds no convoyed Move has no viability that varies: by the classification above it is a pure cycle, and it takes the all-succeed policy (a test holds it), which both engines share verbatim (`common.commit_cycle_policy`). The all-succeed policy is the classic rule on clean rotations, deterministic and capacity-consistent even where no stable completion exists. The policy is seeded with the residue only: orders decided beforehand, orders on which all stable completions agree (step 2 of the selection), and, where there is no stable completion, orders to which Eval gives one verdict under every completion (step 3) are decided before the policy runs (Sec. 6). The stronger form holds too: not just the policy but the whole **selection** is shared code (`common.resolve_residue`). Both engines settle a residue of at most twenty orders (the enumeration bound, defined in Sec. 6) by exact enumeration of its stable completions, so the policy only ever sees genuine ambiguity. An order that the resolver's propagation leaves open is not evidence of genuine ambiguity, since propagation splits cases only inside one admission (Sec. 6) and can stall on a component with a unique stable completion. For example: propagation can stall on the position W of Sec. 8.4 with its Dislodge also supported by a 2A, which then has strength 4. The enumerator, which implements this section literally, remains the fallback certainty.

6 The algorithm (five phases)

This is my five-phase design: Phase 0 parses and validates the sheets, Phase 1 builds the dependency graph of the orders, and Phases 2–4 are the **stages of one propagation**: a monotone evaluation of the undecided relocating orders, iterated together with an in-line commit of vacancy cycles until neither decides anything more.

Phase 0 – parse & validate

Phase 0 parses the sheets against the ruleset's order registry and validates every order statically; an invalid order becomes a Hold (Sec. 1). It is deterministic: it reads no clock and draws no random number, its result does not depend on the iteration order of any dictionary, and the sheets stay ordered.
An order is *legal* iff the board as it stands allows it, read without any other order (DATC 4.E.1): its type is in the ruleset's order registry; in Beleriand its unit spec fits some unit

of its power in its province (`strict_designation`, Sec. 7); a Move, a Dislodge or a Bar passes its static check (Sec. 1); a support's supporter reaches its target by the reach rule of Sec. 1 or stands in its target and the target is an Area (Sec. 1), and its subject reference, read without its designator, describes a unit standing in the province it names and is not ambiguous: the units other than the supporter that it describes there agree in power, kind and base strength (Sec. 1); a Convoy's unit is a fleet in a sea province, and its cargo reference describes an army that fleet can carry (Sec. 3.5). Whether a unit standing where a Dislodge's or a Bar's victim reference writes answers to that reference is not checked here: a reference that describes no unit does not make the order illegal, and where the order is legal it stays valid and fails (Sec. 3.4, Sec. 4.2).

What a Support or a Convoy reads from the other orders is checked once every unit has its order: whether its subject received the subject order, and whether its cargo's Move goes by sea to the written destination. A support also fails that check where its subject reference, read with its designator, describes none of the units that it describes without it (Sec. 1), and where its subject is a convoyed Move whose army the fleets standing on the board, whatever their orders, could not carry from its province to the Move's destination (Sec. 1) once the supporting fleet itself is left out (DATC 6.D.31). Another fleet standing in the supporter's own sea Area stays in, so it may complete the chain. A Support or a Convoy that fails only that check is *invalid*: it keeps its unit, which holds.

Each order is bound to a unit of its power in its province as follows. Where its spec fits a unit there that no earlier legal order has bound, it is bound to the first such unit. Where its spec fits no unit there at all, it is bound to the first unit there that no earlier legal order has bound: the written kind and base strength are advisory on the standard map (DATC 4.C.2 choice b), and they make the order illegal in Beleriand. Where its spec fits only units that earlier legal orders have bound, it is bound to none, so that it is passed over. `A` never stands for an army of another base strength (Sec. 1). Of several orders written for one unit the first legal one is its order and the others are passed over (DATC 4.D.3, 4.E.1; the rulebook's 4.C.2 and "Invalid Orders"). So a legal order wins over every illegal order written above it, whatever its type, and over no legal one: a legal Support or Convoy keeps its unit even where it proves invalid. A unit none of whose orders is legal holds.

The output of Phase 0 is the set $\mathcal{O}$ of Sec. 1: exactly one order for every unit, the order written for it where that order is valid and a Hold where it is not or where none was written. Each is kept with what was written, and so is every order that Phase 0 ignores or passes over, so that every written order can be reported (Sec. 5.1): a replaced order, and an order ignored or passed over, as failing.

Phase 1 – static dependency graph, SCC condensation

Phase 1 builds the order-dependency digraph D that Sec. 5.1 announced. Its vertices are the orders in $\mathcal{O}$; "aimed at" is the relation of Sec. 1. For two different orders $o, o' \in \mathcal{O}$, D has the edge (o, o') from o to o', read "o depends on o'", in exactly the following cases.

- o is a relocating order, and o' is aimed at $\mathrm{dest}(o)$ (the rivals, their supports and their convoys, the supports of the defenders, the Bars at the door), or is the order of a unit standing in $\mathrm{dest}(o)$ (stayers and defenders, the counter-move of an edge contest), or is a support or a relocating order aimed at $\mathrm{prov}(o)$, the own province of o (the supports of a counter-move; whatever may dislodge the unit of o itself, which

prevent-nulling reads), or is a support aimed at prov(d) for a Dislodge d aimed at dest(o) (the supports of the order that meets d in an edge contest, which decide whether d lost it), or is a Convoy of o.

- o is a stationary order of a unit standing in p, and o' is a relocating order, a support or a Convoy aimed at p (what threatens that unit – for a support, its potential cutters – with their supports and their chains), or is a support aimed at prov(d) for a Dislodge d aimed at p (the supports of the orders that meet d in an edge contest, which decide whether d takes part in the engagement of Sec. 4.4), or o and o' are two Convoys of one Move (whether its route runs through the fleet of o depends on the fates of the others).

Lemma L1 (static closure)**.** For every labeling L and every order $o \in \mathcal{O}$, the verdict on o under L – Eval(L)(o) if o is relocating, the derived verdict of Sec. 5.1 if it is stationary – depends on L only at the relocating orders reachable from o in D.

Proof sketch: Case analysis on Secs. 2–4: every resolution-dependent quantity (cut, viability, void, vacancy, admission, prevent-null, the edge contests) that the verdict on o reads is a function of L at the orders that o has an edge to in D and of the same quantities of those orders, so induction along the edges of D gives the claim. No rule reads state outside the orders' provinces.

The graph is over orders, built from their syntax. Lemma L1 is what makes the SCC split knowable **before** supports or convoys are resolved: D overapproximates true dependence, so independence in D implies true independence. Couplings that die during resolution (a cut support, a disrupted convoy) only make components *smaller* than D predicted, never larger. Support and Convoy orders couple distant provinces exactly through their edges.

One dependency does not survive when D is restricted to the open orders of the residue step below, and it concerns the walk groups of Sec. 4.4 only. The open orders are the orders whose verdict or one of whose facts is still UNDECIDED, as defined there, and they keep every other path of D. The engagement of Sec. 4.4 (twins rule 6; `Evaluator._walk_engagement`) gives every attack of a walk group its victim in the pool as *one* function of the verdicts on all the attacks and on the orders of the whole pool, but the edges of D tie the pool together only *transitively through the shared attack*. The relocating order of each twin has an edge to the attack through its own province, and the attack has an edge to every twin, which stands in the province it is aimed at. The enumerator takes the components of the full D, so a walk group always lies in one component and is never split. The resolver instead takes the components of D **restricted to the open orders**. Its residue is smaller by design, as described below. Once the bridging attack is decided during propagation and leaves that restricted graph, twins bound for *different* doors fall into separate residual components while the engagement still reads across all of them, and so reads an undecided twin outside the component, against Lemma L1. The resolver therefore makes the pool and the attacks of each walk group a clique before it takes the components of its restricted graph (`common.residue_adjacency`), which restores the dependency under the restriction. No such edge exists at capacity 1, where there are no walk groups, so the standard map and every DATC outcome are untouched.

The enumerator settles the components of D (Sec. 5.1) one at a time in dependency order. The resolver propagates over all the orders of the phase at once and takes components only for the residue that propagation leaves (below). Neither runs anything concurrently: by

Lemma L1 and Theorem T3 the outcome does not depend on the schedule, so components that do not reach one another could be processed concurrently without changing it. In the resolver, Phases 2–4 run as follows.

Phases 2–4 as monotone stages (the resolver)

Partial labelings. Let a *partial labeling* be a function $\Lambda\colon \mathcal{M} \to \{\text{SUCCEEDS}, \text{FAILS}, \text{UNDECIDED}\}$. Say that an order m is *decided* by Λ iff $\Lambda(m) \neq \text{UNDECIDED}$.
A labeling L (Sec. 5.1) *completes* Λ iff $L(m) = \Lambda(m)$ for every m that Λ decides.
For two partial labelings let $\Lambda \sqsubseteq \Lambda'$ iff $\Lambda'(m) = \Lambda(m)$ for every m that Λ decides: Λ' decides at least what Λ decides, and the same way. Under $\sqsubseteq$ the partial labelings form a finite partial order whose least element decides nothing.

Facts and intervals under a partial labeling. Let Λ be a partial labeling. A fact of Sec. 5.1 – a unit is dislodged, a convoy is viable, an order exists as an attacker – has one of three values under Λ: it is TRUE only if it holds under every labeling that completes Λ, FALSE only if it holds under none, and UNDECIDED otherwise. The converse is not required: the resolver may leave UNDECIDED a fact that is in truth forced, because it computes a fact from uncertain inputs as if they were independent, and splits cases only inside one admission, on the departure of the one occupant or victim that admission reads, or because the engagements of two walk groups read each other through supports given from their pools (Sec. 4.4). That costs a round or a residue, never a verdict.
Let a *strength interval* of a pipeline value v under Λ be an interval $[lo, hi] \subseteq \mathbb{Q}_{\geq 0}$ that contains the value of v under every labeling that completes Λ. Its endpoints are sums over the supports whose delivery is forced, respectively possible, so they come from the finite set of achievable partial sums.
Let $Z \subseteq \mathcal{M}$ be the set of convoyed Moves that the Szykman policy has put out of play (Sec. 5.2): empty until a selection applies that policy (below), and fixed while propagation runs.
A fact and an interval are computed from Λ together with Z – every labeling that completes Λ is read with the Moves of Z and their fleets holding – and are never stored.

Stages and propagation. Let Eval_3 be the function from partial labelings to partial labelings that the three stages below compute together: for every partial labeling Λ and every $m \in \mathcal{M}$, $\text{Eval}_3(\Lambda)(m) \coloneqq \text{SUCCEEDS}$ if the three-valued facts and the strength intervals under Λ settle the contests of m so that m is admitted (Sec. 5.1), $\text{Eval}_3(\Lambda)(m) \coloneqq \text{FAILS}$ if they settle them so that it is not, and $\text{Eval}_3(\Lambda)(m) \coloneqq \text{UNDECIDED}$ otherwise.
I design the stages to make Eval_3 *monotone* – $\Lambda \sqsubseteq \Lambda'$ implies $\text{Eval}_3(\Lambda) \sqsubseteq \text{Eval}_3(\Lambda')$ – and to make it *sound at stable labelings*: $\text{Eval}_3(L) \sqsubseteq \text{Eval}(L) = L$ for every stable labeling L. Equality holds – every fact is two-valued and every interval is a point – wherever no walk group's engagement reads itself again through supports given from a walk group's pool (Sec. 4.4); such an engagement Eval_3 leaves UNDECIDED even under L, and the selection settles it. Under a labeling that is not stable, Eval_3 and Eval can differ: Eval_3 reads the unit of a Dislodge that lost its edge contest as staying, whatever that labeling says. With monotonicity, soundness at stable labelings makes Eval_3 sound for stable labelings: for every partial labeling Λ and every stable labeling L that completes Λ, $\text{Eval}_3(\Lambda) \sqsubseteq \text{Eval}_3(L) \sqsubseteq L$, so a verdict that Eval_3 gives under Λ is the verdict of every stable labeling that completes Λ.
Let the *vacancy commit* V be the function from partial labelings to partial labelings that

gives, in $V(\Lambda)$, the verdict SUCCEEDS to every Move of U_Λ and keeps Λ elsewhere, where U_Λ is the largest set of Moves seeded under Λ (S3, below) that contains, with each of its Moves, every Move left undecided by Λ of a unit standing in that Move's destination. V only decides, but it is not monotone, and its verdicts need not hold in every stable labeling: it is the all-succeed policy of Sec. 5.2, applied in-line to the cycles it finds.
For two partial labelings Λ and Λ' let $\Lambda \lhd \Lambda'$ be the partial labeling that agrees with Λ wherever Λ decides and with Λ' elsewhere.
Let *propagation* be the sequence $\Lambda_0, \Lambda_1, \ldots$ of partial labelings in which Λ_0 decides nothing and $\Lambda_{k+1} \coloneqq V\big(\Lambda_k \lhd \mathrm{Eval}_3(\Lambda_k)\big)$, stopped at the first k with $\Lambda_{k+1} = \Lambda_k$. Call each step a *round*, and that Λ_k the *result*. Every round only decides, so the sequence ascends in $\sqsubseteq$ and stops after at most $\#(\mathcal{M})$ rounds that decide anything (Theorem T2). It needs no monotonicity for that. A round takes from Eval_3 only verdicts that hold in every stable labeling completing the partial labeling it starts from, so by induction over rounds every verdict of the result holds in every stable labeling that agrees with the verdicts V gave. The verdicts V gave settle vacancy cycles the way the all-succeed policy does, and T3a says what that needs. The three stages are the following.

- **S1 – paths**: This stage computes the convoy viability of Sec. 3.5, three-valued, from the three-valued dislodgement of the convoying fleets. Call a fleet *unthreatened* iff no relocating order is aimed at its sea. An overland path is TRUE at once, and so is the viability of a convoy whose fleets are all unthreatened. The cuts such a convoy carries apply at once. Viability stays UNDECIDED until it is forced, and a convoyed Move with its fleets is treated as holding only by the Szykman policy of a convoy-entangled residue (Sec. 5.2).

- **S2 – supports**: This stage computes the one-shot cut arithmetic of Sec. 3, the sum over the qualifying cutters, whose predicate holds the own-power filter and the directed-against exception. Cutter values are pre-cut and pre-void (Sec. 3.2), and support-holds have no protected province (Sec. 3.1). A dislodged supporter's contribution is voided by S3, not by the cut pass, and that of a support that backs a Bar is not voided at all (Sec. 3.3). A cutter a whose existence is undecided – a convoyed one – contributes the interval $[0, \mathrm{cutval}(a)]$. Within a round every order is evaluated against the same Λ, **synchronously** – every evaluation reads the old partial labeling and writes into the new one – so that the provinces could be processed concurrently with a deterministic result, and the one-shot model makes it exact for cuts.

- **S3 – contests**: This stage decides the edge contests **first**, then the displacement of **stationary** units and the voids that follow, and then the entry step, the admission, with **vacancy resolution**. Displacement is that of Sec. 4.2: head-to-head defenders use defend, tied attacks dislodge nobody, and only an admitted winner dislodges. A dislodged unit's support or Convoy is void, but its relocating order still attacks and cuts, and prevents everywhere but where prevent-nulling (Sec. 4.2) sets its prevent to 0; its Bar still bars (Sec. 4.2), and its support of a Bar still counts (Sec. 3.3). Say that a Move m is *seeded* under a partial labeling Λ iff Λ leaves it undecided, it is no Dislodge, it fights no edge contest and no Dislodge could meet it head-to-head (Sec. 4.1), it exists as an attacker and no Bar stops it under Λ, some unit standing in $\mathrm{dest}(m)$ has a Move that Λ leaves undecided, and Eval_3 admits m once those Moves are supposed to succeed. The *vacancy iteration* gives every seeded Move the

tentative verdict "SUCCEEDS (unstable)", withdraws that verdict from any Move one of whose assumed departures is neither decided nor itself still unstable, and cascades until nothing changes. A Move whose verdict is withdrawn goes back to UNDECIDED; it is not decided FAILS. The iteration is a greatest-fixpoint pass whose downgrades terminate, and what it leaves unstable is the set U_Λ that the vacancy commit V decides SUCCEEDS, the Sec. 5.2 cycle policy applied in-line. Edge-contest pairs are **excluded** from vacancy optimism: two opposing non-convoyed Moves between displacement-governed provinces, and a Dislodge with its victim's Move or Dislodge, fight an edge contest (Sec. 4.1), never a rotation. A 2-cycle of plain Moves with an Area at either end is two entries and rotates like any vacancy cycle (BATC.B.6). The commit quantifier is per dependency component, never global. A failed mover's dislodgement is a fact like any other (Sec. 5.1): it is found only once its relocating order fails, though the mover may be found not dislodged before. It voids nothing, since a mover carries no support or Convoy, but prevent-nulling (Sec. 4.2) reads it, so the entries into the province its dislodger came from may be decided only after it. In a multi-unit province – a bounded Area holds up to six units and an unbounded one any number – the contests are the per-object contests of Sec. 4.3 and Sec. 4.2, and Area admission joins the vacancy iteration whenever capacity binds. A full Area's entrants wait on leavers exactly like a capacity-1 province's.

Residue, selection, resume. Suppose propagation leaves relocating orders UNDECIDED. Call an order *open* iff it is an UNDECIDED relocating order, or a decided one whose existence as an attacker or whose unit's dislodgement is still UNDECIDED, or a stationary order with an UNDECIDED fact: the dislodgement of its unit, a support whose strength interval has not closed to a point, a Convoy of a Move whose viability is UNDECIDED, a Bar whose entrant is UNDECIDED.

Let the *residual components* be the strongly connected components of D restricted to the open orders, the pool and the attacks of every walk group having been made a clique first (Phase 1).

The resolver **runs the selection of Sec. 5.1 exactly** on the first residual component that holds an UNDECIDED relocating order, in the order in which Tarjan's algorithm emits the residual components – dependencies first – when it visits the open orders in canonical order (Sec. 1), with the UNDECIDED relocating orders of that component as the residue R. It enumerates the completions of R, takes a unique stable one, decides what several stable ones agree on, or, where none is stable, what Eval gives alike under all of them, and enumerates again, and only genuine ambiguity is classified per Sec. 5.2 and settled by the ruleset's backup policy. Then it resumes propagation, and repeats. The process terminates: each application of the selection decides every relocating order of its residue (Theorem T2). The set that propagation leaves UNDECIDED is not the genuinely ambiguous set: V has already settled the clean cycles by the cycle policy applied in-line, and propagation derives forced facts one-sidedly and splits cases only inside one admission, not, for example, on a targeted Dislodge victim's departure where the branches force opposite verdicts for different orders. So a component with a **unique** stable completion can stall UNDECIDED, and seeding the backup policy on it would unravel the uniquely determined answer. For example: the position W of Sec. 8.4 with its Dislodge also supported by a 2A has one stable completion, which propagation does not reach. In a variant of the ANG position (BATC.B.12) in which the victim's Move bounces off a holding army and neither Dislodge meets it in an edge contest (`tests/engine/test_residue_selection.py`), handed whole to the selection as the enumerator hands its

components, two Dislodges naming a victim that is ordered to leave fail in every stable completion, yet an all-succeed seed would read the victim as departed, and the inflated entry class would bounce a genuinely succeeding Move. The selection is therefore shared code (`common.resolve_residue`), which both engines run unchanged, like the policy realization itself.

Convoy-free fast path (Theorem T4). In the standard instantiation, in components with no convoyed order, one S2 pass and one S3-stationary pass decide every support and every stationary dislodgement, with no iteration between S1 and S2, and the entries left need only the entry rule of S3, repeated along their chains of departures, with the edge-gated vacancy iteration for their cycles: a special case that Theorem T4 argues rather than the general algorithm. It is a statement about the calculus and not a code path: the resolver runs the general propagation on every component and does not take the shortcut, and in Beleriand nothing justifies the shortcut, because Theorem T4 does not extend there (Sec. 8.4).

The enumerator (`csp.py`) – executable spec

The enumerator runs the same Phase 0 and Phase 1. Inside each component of D, taken in dependency order, it decides the two forced facts of Sec. 5.1. The relocating orders of the component that are then still undecided are its residue R, and up to a bound on its size (below) the selection of Sec. 5.1 settles it literally: the enumerator enumerates the $2^{\#(R)}$ completions of R and keeps the stable ones. If exactly one completion is stable, it takes that one. If several are stable, it decides every verdict that the stable completions share and enumerates again over the rest. Unanimity is enumeration's counterpart of propagation, T3a computed exactly. A convoyed Move is decided only when its *viability* is also the same under all the stable completions, since its cut delivery stays Szykman's business whatever its verdict. If none is stable, it decides every order of the residue to which Eval gives the same verdict under every completion (step 3 of Sec. 5.1) and enumerates again over the rest. If none is stable and no order is so decided, or several are stable and unanimity decides nothing, it applies the backup policy to that residue and, where the policy leaves orders undecided, enumerates again. In the second of these two cases, a verdict that the stable completions share on a convoyed Move whose viability varies decides nothing. The policy is seeded with the residue only. The reason is that seeding an order that fails in every stable completion poisons the shared all-succeed commit: read as alive, its prevent overfills a Sec. 4.3 class and the synchronous cascade unravels a live vacancy cycle to all-fail (the ANF position, BATC.B.11). This loop is the shared `common.resolve_residue`, which the resolver runs on its residual components, as described above. I bound the enumeration at twenty orders (`common.MAX_ENUMERATED`). Where the component holds no convoyed Move that Szykman's rule has not settled, a residue of more than twenty orders is settled without enumeration in two cases. If the all-succeed completion of the residue is stable, the selection takes it. The reason is that the verdicts which all stable completions share are then successes, so that completion stays stable as unanimity shrinks the residue, and the all-succeed policy of Sec. 5.2, seeded with successes only, downgrades none of its orders. Otherwise, if the orders of the residue are all overland Moves between provinces of capacity 1 and the set of their destinations is the set of their origins, the all-succeed policy of Sec. 5.2 settles it. On any other residue of more than twenty orders the loop raises an error. The bound sits in that shared loop, so it

binds the residues of the resolver as well (Sec. 8.2). When the loop enumerates again, stability is checked on every order of the original residue, those that unanimity or step 3 has decided as well as those still enumerated. The reason is that a completion stable on the smaller set alone may contradict a verdict already decided, and counting it would pollute the unanimity and the Sec. 5.2 viability sets. Within the bound, the enumerator *is* Sec. 5 by construction. It ships first and remains the differential oracle forever (`resolver == enumerator` on every DATC case, every BATC case and the nightly random positions).

Repairs to algorithm v2

The algorithm of this section is my own five-phase design, algorithm v2, the second version of a first draft, v1. Validating each version on paper against the calculus of Secs. 2–5 – on the DATC's scenario families and on the Beleriand rulebook's worked example – found the eight defects below; each entry says what the design did, the position or the argument that breaks it, and where the calculus stands instead. (R2) and (R3) were defects of v1 that v2 itself fixed at source, (R8) is the one that v2 brought in, and the other five are in both versions, (R5) in two shapes. The positions of the validation are held for both engines by tests: those on the standard map as the anchors of `tests/engine/test_adjudicator_smoke.py`, with a legal supporter wherever the validation's was illegal, which changes no expected outcome, and the rulebook's worked example and the position that tells a sum of cuts from the strongest cut as BATC.A.1 and BATC.A.2 (Sec. 10). The validation and the repairs are original to this project, like the design.
What v2 contributed, the calculus keeps: the vacancy iteration of S3, which closed the gap that v1 left on circular movement (DATC 6.C behavior), the pseudo-province reduction of Sec. 4.2, and the convoys-as-holds idea, which survives as the Szykman policy on a residue (Sec. 5.2). Algorithm v2's phases map onto this section as follows: Phase 0 and Phase 1 stand as they are, its phase 2 is S2, its phases 3 and 4 are S3, and its two rounds are replaced by S1 together with the Szykman policy. Two of its phrases the calculus reads in one way only. Its phases 2 and 3 work "for each province" and say that this "can be done concurrently", which for a deterministic result has to mean that the updates within a stage are synchronous: every update reads the old values and writes the new ones (S2). Its phase 3 decides about "the unit that stands there", while an Area holds several units, so the contests of S3 are per object of Sec. 4.3 and Sec. 4.2.

(R1) The dislodgement test. In both versions, phase 3 sets the strengths of the attacks on a province against the hold strength there and decides from these alone whether the unit standing there is dislodged – in effect, it is dislodged when some attack exceeds its hold. Two positions break the test. In the beleaguered garrison two supported attacks of equal strength both bounce and the defender survives (DATC 6.E behavior), though each attack exceeds the defender's hold; and in a head-to-head the defender resists with defend, not hold (Sec. 4.1), so an attack stronger than the defender's hold can still lose the edge contest. Algorithm v2 kept the test. The calculus dislodges by the displacement contest of Sec. 4.2, which S3 applies to the stationary units after the edge contests: only an admitted winner dislodges, so tied attacks dislodge nobody, and a head-to-head defender uses defend.

(R2) The own-power filter and the cut exception. In v1, phase 2 cuts the supports given from a province by the strength of the strongest attack on that province, whoever makes it and wherever it comes from. Under the classic rules an attack by the supporter's

own power cuts nothing, and neither does an attack from the destination of the supported Move; v1 had neither rule. Algorithm v2 itself fixed this at source: its phase 2 counts only the attacks of a unit whose owner differs from the owner of the unit in the province, made from another province than the one the supported attack tries to take. In the calculus the two rules are conditions 2 and 3 of the qualifying cutters (Sec. 3.1), the exception taken per province in the standard ruleset and per unit in Beleriand.

(R3) The sum. In v1, phase 2 cuts a support by the strength of the strongest attack, while the Beleriand rulebook diminishes it "by the combined Strength of all Support-cutting Attacks". The two part where several attacks cut a support that is stronger than each of them: a support given by a unit of base strength 2 cut by two attacks of strength 1 keeps nothing under the sum and 1 under the strongest attack (BATC.A.2, Sec. 10). Algorithm v2 itself fixed this at source: its phase 2 cuts by the sum of the strengths of the attacks. In the calculus the cut is the sum in both rulesets (Sec. 3.2; the hook `cut_aggregation`, Sec. 7), and at base $\equiv 1$ with no modifiers the sum is the classic binary cut (Lemma L2).

(R4) What a dislodgement cancels. In both versions, phase 3 says of a dislodged unit only that "its order is cancelled", whatever that order is. Read literally, a dislodged unit's relocating order would stop attacking, preventing and cutting; but a doomed unit's relocating order still cuts (DATC 6.D.21; Sec. 3.3), and it still keeps rivals out of its destination short of prevent-nulling (Sec. 4.2). Algorithm v2 kept the sentence. The calculus types the cancellation: a dislodged unit's support or Convoy is void, but its relocating order still attacks, prevents and cuts, its Bar still bars (Sec. 4.2), and its support of a Bar still counts (Sec. 3.3); S3 applies it with the displacement of the stationary units.

(R5) Fixed schedules. The first draft, v1, runs phases 2 and 3 first on the seas and the coasts – on the coasts only the support cuts, leaving out the cuts of convoyed attacks – and processes dislodgements first on the seas; then it runs them everywhere else. That schedule is unsound, because a coastal support can feed the sea battle that decides the convoy that cuts it, and the counterexamples include positions with no paradox in them: an undisputed convoy whose cut, deferred past the sea pass, flips a sea battle. Algorithm v2 replaced v1's sea-first pass with a two-round scheme – it adjudicates once with every convoy as a hold, voids the fleets dislodged under that hypothesis, and then adjudicates for real – which mends that counterexample. The hypothesis is exactly right *as a policy on the undecided residue* (it is Szykman, Sec. 5.2, and v2 derives the Szykman outcome on the classic self-protecting convoy paradox rather than assuming it). Applied globally it remains a fixed schedule and stays unsound both ways: round A voids provably-viable convoys (a convoy whose cut protects *another* convoy's fleet – the unthreatened first convoy's cut is suppressed while the second fleet's battle is settled), and a fleet that survives round A can be dislodged in round B by a cut its own convoy delivered – a phantom attack through a convoy the rules retroactively disrupt (the zero-labeling paradox, where the answer the tests hold is instead that the army fails and delivers no cut). No fixed schedule is sound once convoys couple the strata, so in the calculus Phases 2–4 are the rounds of one propagation, a monotone evaluation iterated together with the in-line vacancy commit (Sec. 6). Propagation keeps v2's hypothesis, scoped: viability stays UNDECIDED until it is forced, a convoy whose fleets are all unthreatened is viable at once and the cuts it carries apply at once (S1), and convoys-as-holds fires only as the Szykman policy of a convoy-entangled residue (Sec. 5.2). In a component with no convoyed order, under the standard rules, the original five-phase pipeline – one pass of phase 2, one of phase 3, then

the entries of phase 4 – is the schedule of Theorem T4, so a fixed schedule suffices there as far as Theorem T4 holds: it is proved for the supports and the stationary dislodgements and argued for the entries (Sec. 8.4). That is why the original algorithm looked right on paper: for the standard rules, wherever no convoy exists, its schedule is the one Theorem T4 argues for, though not every step of its phases is; Theorem T4 does not extend to Beleriand (Sec. 8.4).

Both policies are algorithm v2's two global mechanisms, applied residue-only. Szykman *is* "replace convoys by holds" – restricted to the convoyed Moves whose viability varies; v2's round-A hypothesis is the right idea applied to every convoy instead of those. Likewise the all-succeed policy *is* the optimistic seed of v2's "succeed (unstable)" iteration – restricted to the cycles that reach the policy. I keep both insights and fix only their scope.

(R6) When a mover is dislodged. In both versions, phase 3 decides whether the unit standing in a province is dislodged before phase 4 decides which relocating orders succeed – for a unit ordered to move away as well. A mover's dislodgement waits on its relocating order: a unit whose relocating order succeeds has left and cannot be dislodged, and one whose relocating order fails remains and may be dislodged where it stands (Sec. 4.3), resisting with its bare base (Sec. 4); only where nothing could displace it is it known, before its relocating order is decided, to stay undislodged. Algorithm v2 kept that sequence. In the calculus a failed mover's dislodgement is a fact like any other, read every round (S3): it voids nothing, since a mover carries no support or Convoy, but prevent-nulling reads it (Sec. 4.2).

(R7) The graph of Phase 1. Phase 1 of both versions divides the map into strongly connected components, so that battles that do not interfere can be adjudicated concurrently. The map's adjacency graph would justify nothing: its strongly connected component is the whole board. The idea is right once the graph is over orders, built from their syntax – the digraph D of Phase 1: D overapproximates true dependence (Lemma L1), so its components are known before any support or convoy is resolved and could be processed concurrently.

(R8) Vacancy optimism and the swap. Algorithm v2 brought in the vacancy iteration: in its phase 4 a move that has to wait for the unit leaving its destination succeeds for now, with the status "succeed (unstable)", every unstable move succeeds once all units are unstable, and a move that fails sends the unstable ones to be checked again; the vacancy iteration of S3 is that iteration, formalized. Seeded with a head-to-head pair it commits an illegal swap: of `A ANK - CON` and `A CON - ANK`, each waits for the other to leave, both go unstable, and both succeed. In the calculus edge-contest pairs are excluded from vacancy optimism – two opposing non-convoyed Moves between displacement-governed provinces fight an edge contest (Sec. 4.1), never a rotation – and the commit, which v2 makes once all units are unstable, is made per dependency component, never globally (S3).

One member added after the freeze. The ruleset interface of Sec. 7 gained one member after its freeze: `strict_designation`, the last row of its table, and its advisory default keeps the standard column byte-identical.

7 Ruleset parameterization (the `Ruleset` protocol)

The table below lists everything rule-specific in the sections above as the obligations of the protocol (`src/entente/engine/rules/protocol.py`). The protocol is frozen, and both rulesets back every member of it.

Hook	Standard	Beleriand
`name`	`"standard"`	`"beleriand"`
`base_strength(unit)`	1	1, 2 or 3
`modified_base(role, unit,order,graph,*, disembark=None)`	no modifiers; `disembark` unread	mountains $\times\frac{1}{2}$ (crossing or supporting through, directional; a convoyed Move tolls the crossings from its ordered disembark seas into the landing province – halved iff there is one and all are penalized); fortress +1 hold if non-relocating order
`cut_exception_scope`	`DIRECTED_PROVINCE`: the attacker from the directed-against province (support-moves only; convoyed too)	`DIRECTED_UNIT`: the directed-against **unit**
`cut_ aggregation(cutvals)`	sum (degenerates to binary at base $\equiv 1$)	sum of all qualifying attacks
cut-target selection (Sec. 3.4, Sec. 4.4) – no hook, shared calculus code	trivial: cap $\equiv 1$, a Move cuts the one support standing in its destination	victim-power sheet order; attack-sheet-order assignment; a mis-specified Dislodge (no unit has the named victim or order, Sec. 3.4) fails and cuts nothing
`order_registry`	Hold, Move, Support and Convoy	the four core types, Dislodge (targeted) and Bar (targeted)
`admission_policy`	greedy descending prevent classes, stop at first non-fitting class (the classic bounce at capacity 1)	same rule, in Areas of capacity 6 and in the unbounded ones
`implicit_displacement (capacity)`	implicit at every capacity: top candidate vs defender (the maps have cap $\equiv 1$)	implicit only at capacity 1 (ordinary provinces and off-map boxes behave classically); in an Area targeted only (Dislodge)
`edge_contest(move_a, move_b,cap_a,cap_b)`	both endpoints displacement-governed (capacity 1), neither Move convoyed	same (Areas are not); a Dislodge meets its victim's Move into, or Dislodge of a unit in, the Dislodge's own province whatever the capacities, in shared calculus code (Sec. 4.1)
`convoy_ capability(fleet, cargo)`	any fleet	the fleet's base is at least the army's; a Morgoth unit is carried by Morgoth's own fleets only

Hook	Standard	Beleriand
`entry_permission(unit, province)`	always	the unbounded Areas Angband and Eriador are power-restricted and one-way; only Morgoth's specials, of base strength 3, enter or re-enter Angband
`backup_policy(residue)`	a pure cycle: the all-succeed policy of Sec. 5.2, which seeds every relocating order as succeeding and fails each one that Eval refutes; convoy-entangled: Szykman (Sec. 5.2)	same (the Szykman half is the rulebook's 4.A.2 choice F; on vacancy cycles the rulebook is silent and the classic rule applies)
`update_supply_ownership(graph,state, ownership)`	after the Autumn retreats an occupied supply center passes to its occupant's power	Area control: to the only power present, or to the one with at least two more units there than any other
`power_symmetric`	True	False (asymmetric powers; sheet-sequence semantics)
`strict_designation`	False – advisory prefix (4.C.2 choice b), and a designator is ignored in matching	True – a kind or base strength, written in an order's own unit spec, a support's subject or a convoy's cargo, that describes no unit standing there makes the order illegal: it is ignored, and the unit takes its next legal order or holds (Sec. 1); a retreat so written binds no dislodged unit (the rulebook's 4.C.2 choice C)

One row names no member of the protocol: cut-target selection is calculus code shared by both rulesets (`adjudicator/common/tables.py`), keyed on cap. At capacity 1 a Move cuts the one support standing in its destination, and in an Area only a Dislodge cuts, by the selection of Sec. 3.4. So the standard column is the same code with nothing to select.

The keyword argument of `modified_base` is a convoyed Move's set of ordered disembark seas (Sec. 2): the calculus computes it for every ruleset from the written convoy orders, and it is `None` for every other order.

Two rows lie outside the movement calculus: `name` is the ruleset's registry key, on which the adjudicator never branches and by which the game loop finds the ruleset's game hooks, and `update_supply_ownership` is the ownership update at the season boundary after the Autumn retreats, before the adjustment of Sec. 9, which reads its result.

The Beleriand column is the Beleriand ruleset's movement calculus; its game layer (Sec. 9) works around it. In a Beleriand game the adjudicator receives the Beleriand ruleset wrapped by the game layer (`src/entente/variants/beleriand/layer2.py`), and of the members that the adjudicator calls, the wrapper answers every one as the Beleriand column does but `entry_permission`, which also reads the state of the game.
Besides the entries that the column refuses, the wrapper's `entry_permission` refuses three

more. It refuses a devastated province to every unit of a power other than MORGOTH unless that unit or one of its twins (Sec. 4.4) carries a Dwarven mask (the rulebook's "Glaurung Unit and Devastation"); it refuses `FOR` and `FON`, the Girdle of Melian, to every power but DORIATH and the powers that have gained leave to enter it (Sec. 9); and it refuses `GON`, while GONDOLIN's shield stands, to every power but GONDOLIN (Sec. 1). Phase 0 and the retreats read it, so such an entry is illegal in a movement phase (Sec. 1) and closed to a retreat (Sec. 9).
In a movement phase each Move or Dislodge into devastated ground that Phase 0 keeps for a unit of a power other than MORGOTH claims, in sheet order, one unit that carries a mask among that unit and its twins, and no unit is claimed twice. An order that finds no such unit left is voided before the adjudicator runs, like an illegal order, and its unit takes its next legal order or holds (Sec. 1).

In the last row the setting decides more than validity. Under strict designation an order whose written kind or base strength describes no unit of its power in its province is illegal, so it gives way to a later legal order for the unit it would have bound, and an order whose spec fits only units that earlier legal orders have bound binds none (Sec. 6). Under the advisory setting an order whose written kind or base strength describes no unit of its power in its province binds the unit of its province that no earlier legal order has bound, its written kind and base strength being advisory (Sec. 1), and an order whose spec fits only units that earlier legal orders have bound binds none, as Phase 0 says (Sec. 6).

Acceptance criterion. The DATC passes **through this table**: the engine contains no `if ruleset == "standard"` branches, and the standard column is data, not code paths.

8 Theorems

8.1 T1 – degeneracy (the standard instantiation: strength 1, capacity 1)

Theorem T1 (degeneracy)**.** Let the *standard instantiation* be the calculus with base $\equiv 1$, no modifiers, cap $\equiv 1$, implicit displacement, edge contests on, numeric one-shot cuts with the standard exception, the core registry, and the policies all-succeed and Szykman (the standard column of Sec. 7). The standard instantiation adjudicates classic Diplomacy: on every position of a map with cap $\equiv 1$ its adjudication (Sec. 5.1) is the outcome that the classic rules prescribe, as the DATC reads them under the choices made for this project ("Normative sources").

Lemma L2 (cut degeneracy)**.** Let base $\equiv 1$ with no modifiers. Then for every support $s \in \mathcal{S}$, $\mathrm{eff}(s) = 0 \iff \mathrm{QC}(s) \neq \emptyset$, and therefore, for every s that backs no Bar – every support of the standard instantiation –, $\mathrm{delivered}(s) = 0$ iff $\mathrm{QC}(s) \neq \emptyset$ or $\mathrm{unit}(s)$ is dislodged. With the standard qualifying predicate this is verbatim the classic rule – a support is cut iff its unit is attacked by a foreign unit from any province other than the destination of the supported Move, or is dislodged (Sec. 3.3).

Proof: Here $\mathrm{eff}_0(s) = 1$ for every support s, and $\mathrm{cutval}(a) = 1 + \#(\mathrm{supp}(a)) \geq 1$ for every relocating order a. So $\mathrm{cut}(s) \geq 1$ if $\mathrm{QC}(s) \neq \emptyset$ and $\mathrm{cut}(s) = 0$ otherwise, and $\mathrm{eff}(s) = \max(0,\ 1 - \mathrm{cut}(s))$ is 0 in the first case and 1 in the second. The statement on

delivered is its definition (Sec. 2) for a support that backs no Bar; one that backs a Bar delivers eff(s) even when its unit is dislodged (Sec. 3.3).

Lemma L3 (capacity degeneracy)**.** Let cap $\equiv 1$ with implicit displacement. Then for every province $p \in P$ the entry rule of Sec. 4.3 admits at most one candidate: with d the stayer that remains in p, if any, a candidate m is admitted iff it wins every edge contest it fights (Sec. 4.1), attack(m, d) – prevent(m) where no stayer remains – is strictly greater than the admission rank of every other candidate whose prevent is not nulled at p (Sec. 4.2) and, when d remains, m also wins the displacement contest of Sec. 4.2; if two candidates whose prevent is not nulled tie at the top, nobody is admitted. This is the classic movement rule with standoffs blocking all weaker attacks, beleaguered garrisons surviving, and no self-dislodgement.

Proof: Instantiate Sec. 4.3 with one slot: its capacity-1 form is the rule above, and classes are singletons or top ties. Measuring the candidate by its attack on the stayer is what keeps a support of the defender's own power from carrying an attacker past a rival (DATC 6.E.7), so the defender survives where the classic rule says so.

Status: Theorem T1 is **proved** at the rule-clause level, by Lemma L2, Lemma L3 and the role table of Sec. 4, which mirrors the DATC's own strength definitions. It is **certified empirically** by the full DATC suite through the generalized core (`tests/datc/`, the badge job): any red case falsifies the claim. This dual status is deliberate: the proof argues the mapping, and the suite holds the 150+ behaviors nobody should trust a hand-proof to enumerate.

8.2 T2 – termination

Theorem T2 (termination)**.** Both engines terminate on every input.

Proof: For the enumerator, there are finitely many orders. A residue R has $2^{\#(R)}$ completions. Each pass of the selection ends (a unique stable completion; the all-succeed policy), or decides at least one order of R (unanimity), or takes at least one convoyed Move out of play for good (Szykman), so it runs at most $\#(R)$ plus the number of convoyed Moves of the component times. (Implementation note: one enumeration is capped at 2^{20} completions – `common.MAX_ENUMERATED`; the selection settles a residue beyond the cap without enumeration, where the component holds no convoyed Move that Szykman's rule has not settled, when its all-succeed completion is stable, or when its orders are all overland Moves between provinces of capacity 1, supported or not, and the set of their destinations is the set of their origins, so that the residue is a disjoint union of vacancy rotations, whose Sec. 5.2 outcome the all-succeed policy computes directly; on any other residue beyond the cap it raises a diagnostic error. The cap sits in the shared selection and so binds both engines. The enumerator's residue is a whole component less its two forced facts, and legal positions take it past the cap (Sec. 10). The resolver – the default engine – meets the cap far less often, because propagation and the vacancy commit V leave it small residues. But V seeds no Dislodge (Sec. 6), so a rotation that a Dislodge closes, entering the Area that its victim leaves along the rotation, can reach the selection whole, and with more than twenty relocating orders the selection then takes its all-succeed completion, which is stable.)
For the resolver, the state of propagation is a partial labeling together with the set Z of

convoyed Moves that the Szykman policy has put out of play, which only the selection writes and which stays fixed while propagation runs – a fact or a strength interval is computed from the two and never stored (Sec. 6) – and within one propagation $\mathcal{M}$ has finitely many partial labelings, $3^{\#(\mathcal{M})}$. (The endpoints of the intervals range over a finite set as well: each is a modified base plus, for some of its supports s, a value $\max(0,\ \mathrm{eff}_0(s) - c)$ with c a sum of cut values of qualifying cutters of s.) Every round only decides: it keeps every verdict of the partial labeling it starts from and adds verdicts only for the orders that labeling leaves undecided, through $\lhd$ and through V, which decides only undecided Moves (Sec. 6). Therefore each round either decides at least one more order or is the last, and propagation ends after at most $\#(\mathcal{M})$ rounds that decide anything. Each application of the selection decides every relocating order of its residue, hence at least one, so the resolver alternates propagation and selection at most $\#(\mathcal{M})$ times.

Status: Theorem T2 is **proved**: every round only decides and $\mathcal{M}$ has finitely many partial labelings, so propagation reaches a partial labeling that no round changes. The argument needs no monotonicity, which the vacancy commit V does not have.

8.3 T3 – uniqueness / determinism

Call a component C of D *backup-resolved* iff the selection of Sec. 5.1, run over the components of D in dependency order as the definition of adjudication there prescribes, reaches its step 4 on C, so that a backup policy decides some order of C.
For an order $o \in \mathcal{O}$, let $D(o) \subseteq \mathcal{O}$ be the set of orders reachable from o in D, o itself included.
Say that a labeling L is *stable on* $D(o)$ iff $\mathrm{Eval}(L)(m) = L(m)$ for every relocating order $m \in D(o)$. By Lemma L1 whether L is stable on $D(o)$ depends on L only at the relocating orders of $D(o)$, so labelings stable on $D(o)$ can exist where the position has no stable labeling, for example where it holds, among orders that o does not reach, the position W of Sec. 8.4.

Theorem T3 (uniqueness, determinism)**.** For every position and every order $o \in \mathcal{O}$ from which no backup-resolved component can be reached in D (its own component included), every labeling stable on $D(o)$ decides on o the verdict that either engine returns for it, where that engine returns a result (Sec. 8.2), so the two engines agree on o wherever such a labeling exists; and `adjudicate`, under either engine, is a pure function of the state and the ordered sheets.

Proof: (a) *Propagation soundness.* Every verdict that a round of propagation takes from Eval_3 (Sec. 6) holds in **every** stable labeling that agrees with the verdicts the vacancy commit V gave; this follows by induction over rounds, since a verdict that Eval_3 gives under a partial labeling Λ is the verdict of every stable labeling that completes Λ (Sec. 6). A verdict that V gives settles a vacancy cycle, and the entries that wait on it, the way the all-succeed policy of Sec. 5.2 does; either the cycle lies in a component that the selection of Sec. 5.1 would backup-resolve, so that the claim excludes it and every order that reaches it, or the verdicts of V on it hold in every stable labeling. The selection decides a component without a backup policy only through a unique stable completion or through verdicts that all its stable completions share. By Lemma L1 a verdict reads only what its order reaches in D, so induction over the components in dependency order gives the claim: a component that reaches no backup-resolved one is decided the same way in

every labeling stable on $D(o)$, with o any of its orders. (b) *Determinism.* Propagation is a deterministic iteration – rounds over the undecided orders, each read against one partial labeling, from the partial labeling that decides nothing – so its result is unique. The order in which residual components are selected, the content of the policies, and Beleriand's sheet-sequence tie-breaks are deterministic functions of (state, ordered sheets). Neither engine runs anything concurrently: the enumerator settles the components one at a time in dependency order, the resolver propagates over all orders and then settles its residual components one at a time, and both report the rows in sheet order over the powers in canonical order and the dislodgements in the canonical order of their units. Therefore `adjudicate` is a pure function.

Status: Theorem T3 is **proved** but for one step of (a), which is argued: that every verdict the vacancy commit V gives either holds in every stable labeling or lies in a component the selection would backup-resolve. The induction of (a) rests on the soundness of Eval_3 for stable labelings (Sec. 6), which is not proved stage by stage and which the differential suite checks only through its outcomes. The certifying tests are the determinism property and `resolver == enumerator`, which tests the argued step on every sampled position: a verdict of V that contradicted a component settled otherwise would make the two engines differ. The determinism property adjudicates the same inputs twice and, for the standard ruleset, also under a permuted sheet order, on sheets that write one order for every unit, whose sequence that ruleset then does not read.

8.4 T4 – convoy-free two-pass sufficiency (the pipeline theorem)

Call a support $s \in \mathcal{S}$ *protected* iff a foreign relocating order targets $\mathrm{unit}(s)$ and every such order is the protected attacker of s (Sec. 3.1, condition 3): the one attack on it is the attack it may not be cut by, so only the dislodgement of its unit can still void it.

Theorem T4 (convoy-free two-pass sufficiency)**.** Let the instantiation be the standard one (Theorem T1). In any component with no convoyed order, after one synchronous S2 pass the effectiveness of every support is final, and the only supports whose delivery is still open are the protected ones, whose voiding is decided by displacement contests whose attack strengths are already S2-final; **such voidings cannot chain**. Hence one S2 pass and one S3-stationary pass decide all supports and stationary dislodgements, and the entry contests of Sec. 4.1 and Sec. 4.3 that remain run in a circle only through move/vacancy cycles (Sec. 5.2), which the edge contests and the all-succeed policy handle.

Proof sketch (depth-1 lemma): A support alive after S2 has no qualifying cutter: at base $\equiv 1$ any one of them would have cut it to nothing (Lemma L2). So the only foreign attack that can still target its unit is its protected attacker, and the support is protected or not attacked at all. Suppose the voiding of a protected support s_1 depended on another protected support s_2 propping the attack of the unit t_1 that s_1 is directed against. Then s_2 supports the relocating order of t_1 into $\mathrm{prov}(s_1)$, so s_2 is directed against the occupant of $\mathrm{prov}(s_1)$ – which is $\mathrm{unit}(s_1)$. For s_2 to be *protected and threatened*, $\mathrm{unit}(s_1)$ must attack $\mathrm{prov}(s_2)$; but the one order of $\mathrm{unit}(s_1)$ is the support s_1. Contradiction. Every other strength that a voiding or a stationary dislodgement reads – the defender's hold, a rival's prevent, an attack on a holding unit – is fed by support-holds or by supports directed against a unit that does not attack, none of them protected, so by the first step it is

S2-final as well. Therefore protected-support voiding depends only on S2-final strengths: its depth is 1. (One order per unit is essential.)

The theorem does not extend to Beleriand, and one position shows it; call it W. FINROD orders `A DNN S A RIV DIS MORGOTH A ANF - FSE`; GONDOLIN orders `A RIV DIS MORGOTH A ANF - FSE` and `A FSE - RIV`; MORGOTH orders `A ANF - FSE`, five times `A ANF H`, and `3A(M) ANG - ANF` (the Morgoth unit, named with its designator as every order of a special is, Sec. 1). ANF is an Area of capacity 6 and MORGOTH's six armies fill it; RIV, FSE and DNN have capacity 1. No order is convoyed, no order is a Bar, and no support is cut or void; the victim moves away from its attacker, so no edge contest is fought (Sec. 4.1).

Let g be the Dislodge, let v be the Move of its victim – g names the victim's action, and one army only has that order – let r be the Move from FSE into RIV, and let e be the Move of the `3A(M)`. No order is aimed at DNN, so the support of g delivers 1 under every labeling, and g has strength $1 + 1 = 2$ in each role of a relocating order; v and r have strength at most 1, and e has strength 3.

r succeeds iff g succeeds: RIV has one slot and no other order is aimed at it, so r enters if `A RIV` leaves, and if `A RIV` stays, r may not displace a unit of its own power (Sec. 4.2).

v succeeds iff r succeeds, in the same way: FSE has one slot and no other order is aimed at it, and if `A FSE` stays, it resists with its bare base 1 (Sec. 4), which v does not beat.

Suppose that g fails. Then r and v fail, the victim remains and resists with its bare base 1, and $2 > 1$ with no Bar and no rival Dislodge in the way: g succeeds. Contradiction.

Suppose that g succeeds. Then r and v succeed, the victim has left, and g is no displacement contest any more but a plain entry (Sec. 4.2). ANF keeps five stayers and has one free slot; the candidates are e with rank 3 and g with rank 2; the ladder of Sec. 4.3 admits e and has no slot left for g: g fails. Contradiction.

Therefore no completion of the residue $\{g, r, v, e\}$ is stable. The residue is genuinely ambiguous and holds no convoyed Move, so it is a pure cycle (Sec. 5.2), and the all-succeed policy settles it: from four verdicts SUCCEEDS it takes back g, then r, then v, then e, and nothing moves. That is the adjudication, in both engines (BATC.B.22), and it is not a stable labeling: under it Eval lets g succeed.

A test holds W on the shared selection, together with three controls that have exactly one stable completion each – without the `3A(M)` the rotation goes through, with five armies in ANF all four orders succeed, without the support of g all four fail – and with the mirror image of W in which the dislodger stands in DNN and the rotation runs through LAD, which has no stable completion either. Where the victim moves into the dislodger's own province instead – W without FSE's army and with `A ANF - RIV` for v – the Dislodge and that Move fight an edge contest (Sec. 4.1), which the Dislodge wins with $2 > \frac{1}{2}$, the victim climbing the penalized crossing from ANF into RIV: the victim is dislodged, and the position has exactly one stable completion (BATC.C.14).

Every order of W but the support of g, which depends on no order and is a component of its own, lies in one component of D; that component has no convoyed order, and the entry contests of Sec. 4.1 and Sec. 4.3 that remain in it run in a circle that is not a move/vacancy cycle (Sec. 5.2). So the last clause of Theorem T4 fails in Beleriand, whatever becomes of its clauses on supports.

For the clauses on supports, the first step of the sketch has a Beleriand replacement. Recall that a walk group is a stack of twins together with the unnamed Dislodges whose reference fits it, of which there is at least one (Sec. 4.4).

Lemma L4 (void locality)**.** Let the position hold no walk group. Let $s \in \mathcal{S}$ be a support, let L be a labeling, and let $a \in \mathcal{M}$ be an order with $L(a) = \mathrm{Eval}(L)(a) = \textsc{succeeds}$ that displaces $\mathrm{unit}(s)$ under L (Sec. 5.1). If $\mathrm{eff}(s) > 0$ under L, then a is the protected attacker of s (Sec. 3.1, condition 3).

Proof: Suppose that a is not the protected attacker of s.
The order a displaces $\mathrm{unit}(s)$ and no walk group exists, so a is a Dislodge whose victim is $\mathrm{unit}(s)$ or, where $\mathrm{cap}(\mathrm{prov}(s)) = 1$, a foreign Move into $\mathrm{prov}(s)$: in both cases a targets $\mathrm{unit}(s)$ (condition 1). No power displaces its own unit (Sec. 4.2), so a is foreign (condition 2). $\mathrm{Eval}(L)(a) = \textsc{succeeds}$, so a exists as an attacker under L (condition 4). Therefore $a \in \mathrm{QC}(s)$ under L, and $\mathrm{cut}(s) \geq \mathrm{cutval}(a)$.
Note that $\mathrm{cutval}(a) \geq \mathrm{attack}(a, \mathrm{unit}(s))$. Both values start from $\mathrm{mods}(\mathsf{move}, a)$; $\mathrm{cutval}(a)$ adds $\mathrm{eff}_0(s')$ for every $s' \in \mathrm{supp}(a)$, $\mathrm{attack}(a, \mathrm{unit}(s))$ adds $\mathrm{delivered}(s')$ for some of them, and $\mathrm{delivered}(s') \leq \mathrm{eff}(s') \leq \mathrm{eff}_0(s')$. (If a is a Move of a wave, it is the champion: no other member takes a slot that the champion does not – a supported champion outranks its twins, an unsupported wave is one class, and a Dislodge from the wave's destination meets the whole wave (Sec. 4.4, rule 2) – so both values are built on the same set $\mathrm{supp}(a)$.)
The order of $\mathrm{unit}(s)$ is the support s, which is stationary, so $\mathrm{unit}(s)$ resists a with hold, and a displaces it only if $\mathrm{attack}(a, \mathrm{unit}(s)) > \mathrm{hold}(\mathrm{unit}(s))$ (Sec. 4.2). Moreover

$$\mathrm{hold}(\mathrm{unit}(s)) \geq \mathrm{mods}(\mathsf{hold}, s) \geq \mathrm{base}(\mathrm{unit}(s)) \geq \mathrm{mods}(\mathsf{support}, s) = \mathrm{eff}_0(s),$$

because what support-holds deliver is nonnegative, a hold is never halved, and a support is halved or left alone (Sec. 2).
Therefore $\mathrm{cut}(s) \geq \mathrm{cutval}(a) \geq \mathrm{attack}(a, \mathrm{unit}(s)) > \mathrm{hold}(\mathrm{unit}(s)) \geq \mathrm{eff}_0(s)$, hence $\mathrm{eff}(s) = \max\Big(0,\ \mathrm{eff}_0(s) - \mathrm{cut}(s)\Big) = 0$. Contradiction.

Lemma L4 replaces the first step of the sketch on positions with no walk group. In Beleriand a support alive after S2 may have qualifying cutters – with base strengths 2 and 3 a support survives a weaker cut – but none of them can void it; so the supports whose delivery is still open after S2 are those whose unit is attacked by their protected attacker, less the supports that back a Bar, which nothing voids (Sec. 2). The engagement of a twin stack is outside the lemma: there the unit that an unnamed attack dislodges and the unit whose support it cuts may be different twins (Sec. 4.4), so the dislodger of a unit need not be a qualifying cutter of its support.
The exception for a support that backs a Bar is what the last step of the sketch needs. That step wants the unit a support is directed against to have no order left with which to attack the supporter, and in Beleriand the single order of the unit that a Bar is directed against can be exactly such an attack. FINROD orders `2A DNN DIS MORGOTH A ANF`, supported by `A LAD S 2A DNN DIS MORGOTH A ANF`; MORGOTH orders `2A ANF BAR FINROD 2A DNN - ANF`, supported by `A ANF S 2A ANF BAR FINROD 2A DNN - ANF`: the victim of the Dislodge backs the Bar that gates that Dislodge. Its support is protected, so nothing cuts it, and it backs a Bar, so nothing voids it: the Bar stands at $2 + 1 = 3$ under both verdicts on the Dislodge, stops an attack of $2 + 1 = 3$ because the Bar wins the tie, and the one stable completion is that the Dislodge fails – decided by propagation alone (a test checks it). Were that support lost with its giver, both verdicts would be stable – the Dislodge fails and the Bar stands at 3, or it succeeds, the backer falls and the Bar stands at 2 – and a component with no convoyed order would go to the backup policy through a support.
Whether the clauses of Theorem T4 on supports – one S2 pass and one S3-stationary

pass decide every support and every stationary dislodgement – extend to Beleriand is **open**. What is known comes from a search. Call a genuinely ambiguous residue *void-type* iff some support survives its cut and is void under one of its stable completions and delivered under another – the mark of dislodge-void feedback (Sec. 5.2). Among 100,000 sampled Beleriand positions – two runs of 50,000 of the sampler of the differential suite, which plays the opening forward and so meets MORGOTH's opening stack of six identical armies – 1,264 residues reached the backup policy. None of them was void-type, and none lacked a stable completion: the knot of W, which is built by hand, did not occur in the sample. The sampler writes no designator, so the search never meets a named special. Both engines agreed on every sampled position.

Status: For the standard instantiation the clauses on supports and stationary dislodgements are **proved** as above. The clause on entries has a concrete algorithm (the edge-gated vacancy iteration of S3, Sec. 6) whose claim – that every circle in which the entries left after the stationary pass can run is a clean cycle of length at least 3, which the iteration commits – is argued (Sec. 5.2) but not mechanically checked. So Theorem T4 as a whole is a **conjecture**, proved in part and argued in the rest, and it has no certificate of its own: the engine implements no two-pass schedule – the resolver runs the general propagation of Sec. 6 on every component – so no test compares such a schedule with the enumerator. What the differential suite certifies on convoy-free positions is that both engines reach the same verdicts, not how many passes that took.

9 Retreats and builds (outline)

Retreats. The retreat phase is a smaller adjudication under the entry rule of Sec. 4.3: every retreat order has strength 0 (both rulebooks: Standard – no strengths at all; Beleriand – "Retreat Orders always have 0 Strength"), and no retreat order is supported. The only retreat orders are a plain Move and a Disband. Say that a retreat order is *legal* for a dislodged unit u iff it is a Disband, or a Move into a province that u reaches and may retreat to (below), its coast read as in a Move of the movement phase (Sec. 1); a Move written `via convoy` is legal for no unit. Each retreat order binds a dislodged unit of its power in its province that its spec describes – the written kind and base strength advisory on the standard map and exact in Beleriand, as in a movement order (Sec. 1, `strict_designation`) – and that it is legal for; an order legal for no such unit is ignored, as an illegal movement order is (Sec. 1), and of several orders for one dislodged unit the first legal one counts. Where several such units are twins (Sec. 4.4), identically worded orders go to different twins (the rulebook: "identically worded Retreat Orders of twins go to different twins, each to a twin for which it is valid where one is free"). The orders of a power are matched to its dislodged units line by line, each line to a twin it is legal for, and a line may take a twin that an earlier line holds by moving that earlier line to another twin it is legal for, along a chain of such exchanges if need be. So a later line never costs an earlier one its unit, and as many lines as can be are bound; a line repeated with no unit left for it is passed over. In Beleriand a retreat order whose spec writes a designator binds only the dislodged special that the designator names, and only where it is legal for that special, by that special's own dislodger: the Morgoth unit and Glaurung are never twins (Sec. 4.4). A dislodged unit that no order binds disbands. In a Beleriand game the retreats read the movement phase's result as the game layer leaves it

after the phase's end-of-phase steps: a fleet that sailed into the West has left its slot, and a Transform has changed its unit's kind. The dislodged units also include every unit that a Devastate expels from the province it stands in, all but MORGOTH's and those that carry a Dwarven mask (the rulebook: "are Dislodged and must Retreat (or Disband) in the following Retreat Phase"). Such a unit has no dislodger, so no dislodger's origin is closed to it, and the row of its own order keeps the adjudicator's verdict.

Let a province be *embattled* iff, in the movement phase, two or more relocating orders aimed at it were live – each existed as an attacker (Sec. 3.1, condition 4) and was not prevent-nulled there (Sec. 4.2), a Dislodge counting as a Move does, whether its victim remained or left, or it has none, unless it lost or tied an edge contest with its victim's order (Sec. 4.2) – and none of them succeeded. At capacity 1 an embattled province is the classic standoff province – left vacant by the standoff, or still held by a unit that stood, which no retreat can enter anyway; in an Area it holds even where slots stay free, and two Dislodges that fail in one Area embattle it whether they stood each other off against one victim or each lost to its own victim's hold.

A unit may not retreat to its own province, nor to its dislodger's origin **unless the dislodger came by convoy** (DATC 4.A.5 choice B), nor to an embattled province, nor to one that its entry permission closes (Sec. 4.3), nor off legal terrain. All of these are facts of the movement phase but the entry permission, which in a Beleriand game the game layer reads when the retreats are adjudicated. There it closes more than the unbounded Areas: the Girdle of Melian, the provinces `FOR` and `FON`, to every power but DORIATH and the powers that have gained leave to enter it (the rulebook: "A Doriath's Player may also give a written permission for other Power to enter the Girdle of Melian"); devastated ground to every unit but MORGOTH's and the units that carry a mask; and `GON`, while its shield stands, to every power but GONDOLIN (Sec. 1). So a written permission given during the retreats can open a destination; nothing ordered during the retreats opens or closes one. In a game the retreat phase is offered only where some dislodged unit has a retreat legal for it into a province with a free slot under the permissions in force when the movement phase ends; otherwise the game adjudicates the retreats at once with empty sheets, so that every dislodged unit disbands and a permission given afterwards opens no destination for it. Having no free slot does not bar a retreat into a province: such a retreat is legal, binds its unit and fails in the entry contest, and the unit disbands even where a later line of its sheet names a free province. Two retreats into one slot disband both.

The capacity-aware admission rule governs retreats too (the rulebook: "During Retreats this rule also applies"): all retreats into an Area form the **single strength-0 class**. The class fits the free capacity whole or every member disbands, and the ladder's stronger-first priority is inert. Admission classes are order strengths, as in movement, not the base strengths of the units. BATC.G.3 checks the choice.

When the Fall retreats are settled, or skipped, the ruleset's `update_supply_ownership` passes the supply centres to their new owners, before the adjustment, which reads its result. In a Beleriand game the game layer's wrapper of that member gives a devastated supply centre only to MORGOTH or to a power that has a unit carrying a Dwarven mask there, and otherwise leaves the centre with its owner (the rulebook: "A Unit equipped with Dwarven Mask may capture a Devastated Supply Centre.").

Builds. Builds follow each ruleset's build and disband rules; under the standard ruleset a power builds in its home centers. In Beleriand the game layer's state – the double-unit prerequisites, the fleet-knowledge gates and the schedules of Men's migration and of

Eriador – gates the same adjustment orders, Build, Disband and Waive, and Beleriand adjudicates its own winter through the game hooks. The strength calculus plays no part: adjustments are order-sheet business.

The sheets are handled as the choices on DATC 4.D say. When a power submits a second order set, the later set replaces the earlier one (DATC 4.D.1); the replacement is the game loop's business, before any adjudicator sees a sheet. Within a sheet the orders are read top to bottom, the first legal ones use up the quota, an illegal order uses none of it, and an order beyond the quota fails (DATC 4.D.4, 4.D.5 and 4.D.6, choice B). A Disband binds a unit of its power standing in its province; in Beleriand the base strength it writes has to be the unit's, an unwritten one meaning 1, and on the standard map the written base strength is advisory (Sec. 1, `strict_designation`). Identically worded Disband orders remove different twins, and a Disband repeated with no unit left for it is passed over (DATC 6.J.2). A Waive uses one build, and builds left unordered are waived. Removals left unordered fall to civil disorder (DATC 4.D.8): the standard ruleset removes the unit farthest from the power's nearest owned center, fleets before armies at equal distance, then by the province's name, and it counts the distance in steps over one graph for both kinds, the union of the army and the fleet adjacencies projected to provinces. Beleriand applies its rulebook's own law, which counts an army's distance over every territory, seas included, and a fleet's over the fleet lanes alone, ties going by territory name. Where a power owns no coastal centre, the fleet lanes lead its fleets to none of its centres, so its fleets are removed before its armies, the farthest of them first by an army's distance. MORGOTH's two specials need no supply (the rulebook: "Morgoth and Glaurung Units don't need Supply Centres"): neither counts among the units that the quota weighs against the supply centres, a Disband that names one fails and uses none of the quota, and civil disorder never removes one.

Say that a centre *may bind* a Double Unit at a winter iff its power has owned it since the previous year's Autumn boundary at the latest, no unit of another power entered it during the year, and it has a supply point that no standing unit holds (the rulebook: "A Player may Build a Double Army after a Year in which no other Power has entered the Supply Centre, and the Supply Centre was Owned by the Player for the whole Year, and the Player has not Built a single Unit supplied by that Supply Centre"). So Beleriand's double-unit prerequisite counts **supply points**, not just years: every supply point supplies at most one standing unit at a time – a province with two Supply Centres has two points – and a point already supplying a unit cannot bind a Double Unit.

The game layer's ledger, which the game keeps beside the state (Sec. 1), remembers which point supplies each unit (the rulebook: "Every Supply Centre supplies one Unit at a time"). A Double Unit is supplied by the point of the centre it is bound to; its build may name that centre, as in `Build 2A FOR BOUND NEB`, and otherwise takes the first centre in canonical order (Sec. 1) that may bind it. A starting unit is supplied by the centre it starts on where its power owns that centre with a point to spare. Every other unit – the specials aside, which need no supply – is supplied by the newest centre that its power owns with a point to spare; ties between equally new centres are broken in canonical order, and a Men unit takes first the centre it is built in where that centre has one. A dead unit frees its point; where one of two identical twins dies, it is the one that frees the older centre. Twins that carry different numbers of Dwarven masks are told apart, so that each keeps its own point and none trades it, and where the game layer chooses which of several matching units a removal takes, it takes one with the fewest masks. A unit whose centre is lost takes another in the same way, at the Autumn boundary and again after

the winter's disbands, while a Double Unit whose centre is captured is disbanded at the Autumn boundary, as soon as the centre passes to its captor and before the adjustment counts the power's units. Where no centre of its power has a point to spare, a unit whose centre is lost stands unsupplied until a point frees: the adjustment removes units by its quota alone, never for want of a point. So a unit is moved to the centre that its power gained last, and the centres that the power has held longest – those that have waited out their year for a Double Unit – are the last to be spent.

10 Implementation mapping

`adjudicator/common/` – what both engines share
Both engines share Phase 0 (`phase0.py`) and Phase 1 (`phase1.py`). They share the static tables, with the victim binding of Sec. 3.4 and the twin machinery of Sec. 4.4 (`tables.py`), beside which sit the walk-group cliques of Sec. 6 that only the resolver reads (`common.residue_adjacency`). They share Eval and the derived verdicts of Sec. 5.1 (`evaluator.py`); the selection of Sec. 5.1, the classification and the two policies of Sec. 5.2 (`selection.py`); and result assembly, with the embattled provinces of Sec. 9 (`assembly.py`).

`adjudicator/csp.py` – realizes Sec. 5 literally
The enumerator: it drives the selection over the components, decides the two forced facts and calls the shared selection. It is the executable form of the specification, it was the first adjudicator to pass DATC 6.A–6.E, and it stays as the oracle.

`adjudicator/resolver.py` – realizes Sec. 6
The resolver: it computes the staged fixpoint – propagation, the vacancy iteration, the residual components – and it is the default engine.

`rules/protocol.py` – realizes the Sec. 7 table
The `Ruleset` protocol, member for member: it is frozen, and both rulesets back every member.

`rules/standard.py` – realizes the Sec. 7 standard column
It consists of pure data and strategy objects.

`variants/beleriand/ruleset.py` – realizes the Sec. 7 Beleriand column
It consists of data and strategy objects again; the sheet-sequence selection of Sec. 4.4 is not among them – it is shared calculus code (`adjudicator/common/tables.py`).

`tests/datc/` – the Theorem T1 certificate
It is encoded from the DATC document only, and its choices among the DATC's alternative outcomes are listed in `docs/datc-choices.md`.

`tests/batc/` – Beleriand conformance
BATC.A.1 is the rulebook's worked example: it has three Moves and two supports, and all five orders fail under the arithmetic of Sec. 3. BATC.A.2 is the sum-vs-strongest discriminator: two cutters of strength 1 leave a support given by a unit of base strength 2 with 0, not 1. Other cases check the bar tie, the mis-specified Dislodge, the multi-Dislodge standoff, dislodger slot accounting and the clash of

a Dislodge with its victim's order (cases that check Sec. 4.1 and Sec. 4.2), greedy overflow (Sec. 4.3), the full-Area vacancy cycle, the fortress forfeit, the mountain fraction and the double-fleet convoy.

`tests/properties/` – the Theorem T2/Theorem T3 certificates
The tests check determinism, unit conservation with the capacity invariant, and `resolver == enumerator`. Determinism is checked with the same inputs twice, and for the standard ruleset also under a permuted sheet order, on sheets that write one order per unit, whose sequence that ruleset then does not read (Sec. 1). `resolver == enumerator` is checked on random positions of the standard map, on convoy-rich ones, on the Beleriand map, and on a synthetic map with a mixed coast (Sec. 2). No test compares a two-pass schedule with the enumerator, so Theorem T4 has no certificate of its own (Sec. 8.4).

Failure containment. The resolver is the default engine and the enumerator its oracle. Nothing falls back from one to the other automatically. Sec. 5 and Sec. 6 are two forms of one specification, so choosing the enumerator instead costs performance, not correctness, as far as it answers. A residue beyond 2^{20} completions raises a diagnostic error rather than enumerate (the cap of Sec. 8.2) when the component holds a convoyed Move that Szykman's rule has not settled, or when its all-succeed completion is not stable and its orders are not a disjoint union of vacancy rotations of overland Moves between provinces of capacity 1, supported or not. For example: on the standard map, 21 armies each ordered into the province of the next, the last onto a unit that holds, form one component of 21 undecided Moves, which the resolver settles by propagation.

Provenance

The calculus, the five-phase algorithm, both instantiation tables, and the theorems are original to this project. The DATC document is used as specification and certification target; its strength-role vocabulary is adopted for Sec. 4 with citation. Kruijswijk's adjudication mathematics is related work: consulted for none of the above, cited for the prior art of fixpoint-style adjudication semantics.